

Elysian Fly
Crimson & Cream
Haley Raburn, Chisanshi Chiwele, Josephine Nyachwo, Max Gottwald, Kalen Lee
MIS 3353 – Database Management
Professor Swetha Siripurapu
3/25/2024



CRIMSON & CREAM

Achieve as one excel as a team!

Executive Summary

With a growing market and an increase of 20.4% year-on-year growth, Elysian Fly Company is dedicated to continuing this growth and embarking on a mission to recreate their decision-making process. After acknowledging the benefits of modern technology in terms of increasing productivity and customer value, Elysian wants to simplify operations by putting in place an Entity Relationship Diagram system. The goal of the proposed project is to create an ERD (Entity Relationship Diagram) specifically for Elysian Fly. With the use of the cutting-edge database system we will develop, Elysian hopes to boost decision-making, increase operational effectiveness, and ultimately produce more profit.

Strategies Implemented:

1. **Comprehensive Data Management:** This was the first step we took towards achieving our goals. Our database offers a strong platform for storing, organizing, and retrieving data such as customer details, fly details, sales trends, and market trends. This database promotes clear and consistent data across all departments, enabling a well-informed decision-making process.
2. **Data Integration:** To bring disparate data sources together into a single, centralized location, we devised and executed a thorough data integration plan. We were able to create a single source of truth by integrating data from several departments and systems, which enhanced data consistency and quality and allowed for improved decision-making.
3. **Poor Structural ERD Design:** Our database project initially had a subpar ERD architecture that was marked by inconsistent, redundant, and wasteful relationships. The lackluster design significantly impeded query performance, data integrity, and overall system stability, which made the project unfeasible.

In conclusion, our database project is evidence of the tenacity, inventiveness, and dedication to quality of our team. We used adaptable and strategic techniques to overcome barriers and produce a reliable and scalable database solution, even though we had difficulties with ERD architecture and iterative development. Overall, we spent \$2,710 on this project and we are optimistic that when we show our database project to potential investors, they will see the creativity and promise of our ground-breaking approach to fostering corporate transformation and competitiveness in the digital age.

Contents

Executive Summary	2
Get to Know the Team: (Crimson & Cream)	4
Conceptual Design.....	7
The Client Meeting	7
Q&A During the Meeting & Information We Learned	7
Significant Assumptions	9
What is an ERD? Why is it necessary?	9
Business Cycles Used	10
Data Provided by the Client.....	11
ERD Created	12
Changes made to generic ERDs	13
Logical Design.....	16
Normalization.....	16
Normalization of Data Provided by the Client.....	16
Normalized Relations.....	16
Differences between ERD and Normalized Relations	20
Referential Integrity.....	20
Physical Design and Implementation.....	22
Data Dictionary.....	22
Denormalization	22
Implemented Physical Design.....	23
Strengths and Weaknesses Encountered During Implementation	23
Specific SQL Statements Requested	24
Three Additional Queries	31
User Documentation.....	34
What We Learned Throughout This Process	40
Appendix.....	42
Team Contract	42
Team Members	42
Data Dictionary Model	44
Project Management	50

Get to Know the Team: (Crimson & Cream)

Haley Raburn

Management Information Systems- Junior

I am from Owasso, Oklahoma, and a lifelong sooner fan. I do not currently have any internship experience but hope to find some this summer. I chose to major in MIS after finding an interest in it through classes that I've taken.

Kalen Lee

Management Information Systems- Junior

I am from Ardmore, Oklahoma and decided to come here because I am a big sooner fan. I am currently interning at the University College as a data analyst. I help build reports and research about freshman student retention.

Maximilian Gottwald

Management Information Systems- Junior

I am from Denton, Texas and decided to come to OU because my dad came here. I am 21 years old and have had 1 internship so far. That internship was with Builders Alliance LLC, in Fort Worth Texas. There I worked as logistics coordinator for the company conducting shipping routes, shipping times, working with other companies over the phone, and managing the allotted time slots for deliveries.

Josephine Nyachwo

Management Information Systems- Junior

I am from Uganda in East Africa, and I joined OU in 2021 after finalizing high school at a UWC Atlantic in Wales, UK. Currently, I work as an office assistant at OU Housing where I utilize my communication skills to help students, parents, and staff with answers to questions regarding on campus housing options. Although I haven't had an internship opportunity, I hope to find one in the upcoming summer.

Chisanshi Chiwele

Management Information Systems- Sophomore

I am from Lusaka, Zambia. I chose to come to OU in 2022 after hearing about the community and the programs offered. Currently, I work part-time at OU IT as a service delivery specialist, where I leverage my communication and problem-solving skills to assist faculty, staff, and students with their technology needs. While I don't have formal internship experience yet, I'm highly motivated to gain practical skills in business technology through an internship.

Conceptual Design

A database's conceptual design serves as its foundation. It entails high-level planning of the database's structure and organization, including attributes and entities, as well as their relationships, using an entity relationship diagram. The database's structure is determined by the information provided by the client and the goals for which the database is intended. This section discusses the stages done during the conceptual design phase and how they were implemented in the database. These steps include gathering additional information from the customer, selecting business cycles to use, and modeling the entity relationship diagram.

The Client Meeting

The client meeting was held on the 8th of March 2024 on Zoom. During this meeting, the interviewers Haley Raburn, Chisanshi Chiwele, Kalen Lee, Josephine Nyachwo and Max Gottwald met up with Sam Ferreira, the owner of Elysian Fly. This meeting was held to ask for more information that wasn't necessarily specified in the case study provided and could help us better understand the demands of the project. This next section goes over questions asked in the meeting and what was learned about the project.

Q&A During the Meeting & Information We Learned

1. Are there any specific requirements the products must meet? If so, do the requirements differ between fly types?

Yes, there are specific requirements for each product made. No, the requirements do not change on different fly types.

2. Are there specific types of discounts and what are they based on?

- *Percentages or dollar amount*
- *Can only use specific discounts once...can't use multiple discounts at once.*
- *Military, student, special*

3. Should there be a minimum or maximum amount of each fly type in stock? (pattern, size, etc.)

- *Inventory on hand.*
- *Find a reorder point based on how common it is.*

4. What are the criteria for discontinuation of products? (low sales, seasons, etc.)

If artisans aren't selling them anymore

5. Do you have employee performance metrics, should they be tracked?

Yes

6. Should we implement in the schedule available guides and their max party preference?

- *10 or bigger groups = more tour guides*
- *Volunteers can be multiple types of examples (like previous employees) that can be tour guides, but they must be classified!*
- *Business determines guide size.*

7. Do you want the favored guides (the ones most picked by customers) to be tracked by how many times they've been chosen?

Guide performance needs to be tracked (potential future employees)

8. Do you want to keep track of the historical record of vendor- manager managed which vendor and the time frame?

Yes, we have vendor manager managing vendors.

9. Could you provide more details on the current method of handling online orders?

- *Online orders we have a third-party company which they're working with currently.*
- *Have sales don't know who the customer is.*
- *That's all they're tracking*

10. How do I track the information about the specific fly pattern, sizes and colors currently marked?

- *We can have several patterns.*
- *Listed on the use case.*
- *Come up with patterns.*
- *Reds and pinks and yellows*
- *Can get creative on the types*

Significant Assumptions

An assumption in an ERD (Entity Relationship Diagram) is a statement that is taken to be true by the creators of the ERD. There is no proof of assumptions made during conceptual design of the ERD to simplify the modeling process or fill in gaps about information not given by the client. This section explains the assumptions made in the creation of our ERD and how these assumptions affect the entire ERD. The assumptions are as follows:

1. Volunteers are classified as employees- Volunteers have an EmployeeID and their role as volunteers will be displayed using the reference table 'EmployeeType' under the attribute 'EmployeeTypeName'.
2. Different employee types, such as managers and guides, handle payment transactions
3. EmployeeType includes positions, such as managers
4. Orders are delivered in full and that is why there is one and only one delivery that goes out per sales order
5. Reference table 'EmployeeType' is used for Employee types instead of subtype/supertype for simplicity and commonality purposes
6. Color, size, and pattern reference tables because they have a finite number of attributes

What is an ERD? Why is it necessary?

An entity-relationship diagram, often known as an ERD, serves as a full blueprint for organizing a company's data structure. It visualizes the relationships between many pieces of information, allowing stakeholders to understand complex data structures. ERDs (Entity Relationship Diagrams) are essential tools for improving communication between stakeholders such as consumers, vendors, and employees. They also provide vital documentation for the system, ensuring clarity and consistency in database design, development, and maintenance efforts.

In this project, we are creating a database for Elysian Fly based on information given to us. An example of how an ERD can help in this case is to show the relationship between guides, customers, and fishing trips. Through the ERD, we show that for each fishing trip there must be one guide, and for the guide chosen we require information about the guide, such as their availability, the number of customers preferred per trip, their performance ranking and other information which are referred to as 'attributes'.

Business Cycles Used

Business Cycles are useful starting points in an ERD, they illustrate key components and interactions within a business, helping stakeholders understand the flow of data processes in the business. It also serves as a blueprint for database designers and developers to design and implement a database system that supports the business cycle effectively. We implemented all the business cycles in our ERD to better show the relationships between different stakeholders. These relationships and the importance of each cycle are explained below:

Revenue Cycle- The Revenue Cycle ERD focuses on the revenue-generating parts of Elysian's business operations. Customers, sales transactions, products (fly), payments, and other relevant entities are all part of this cycle, depending on the activities involved. These entities' attributes include client information, product descriptions, transaction dates, payment methods, and amounts. Relationships in the Revenue Cycle ERD show the progression of operations from client queries and orders to product delivery, billing, and payment collection. This cycle creates a database that efficiently handles revenue production processes, tracks sales performance, simplifies client contacts, and allows financial reporting and analysis inside Elysian's business framework.

Production Cycle- The Production Cycle manages Elysian's manufacturing operations, with a focus on the creation of files and DIY bundles. This cycle comprises raw materials, inventory, production orders, finished products and vendors. These entities' attributes include material quantities and vendor information. Relationships in the Production Cycle ERD depict the flow of materials from suppliers to production and final packaging for sale. Tracking material shipments from vendors also enables seamless operations and inventory management for Elysian's DIY goods, which improves supply chain efficiency and overall manufacturing performance.

Expenditure Cycle- The Expenditure Cycle refers to the set of activities within an organization focused on the spending of money. The activities around the expenditure cycle are meant to ensure that the company has the supplies, materials, and services it needs to meet its objectives, without spending more money than it must. The main entities within this cycle are 'PurchaseOrder' and 'PurchaseOrderLine', which entails Elysian making orders to vendors and paying invoices on the orders made.

Data Provided by the Client

Down below we can see the data provided by the client which includes information about various vendors and their products. It consists of three different vendors: 'Stellas Outdoor Shop', 'BigRigFishing', and 'BoomerSoonerFly'. Every vendor has different products, costs, patterns, sizes and colors.

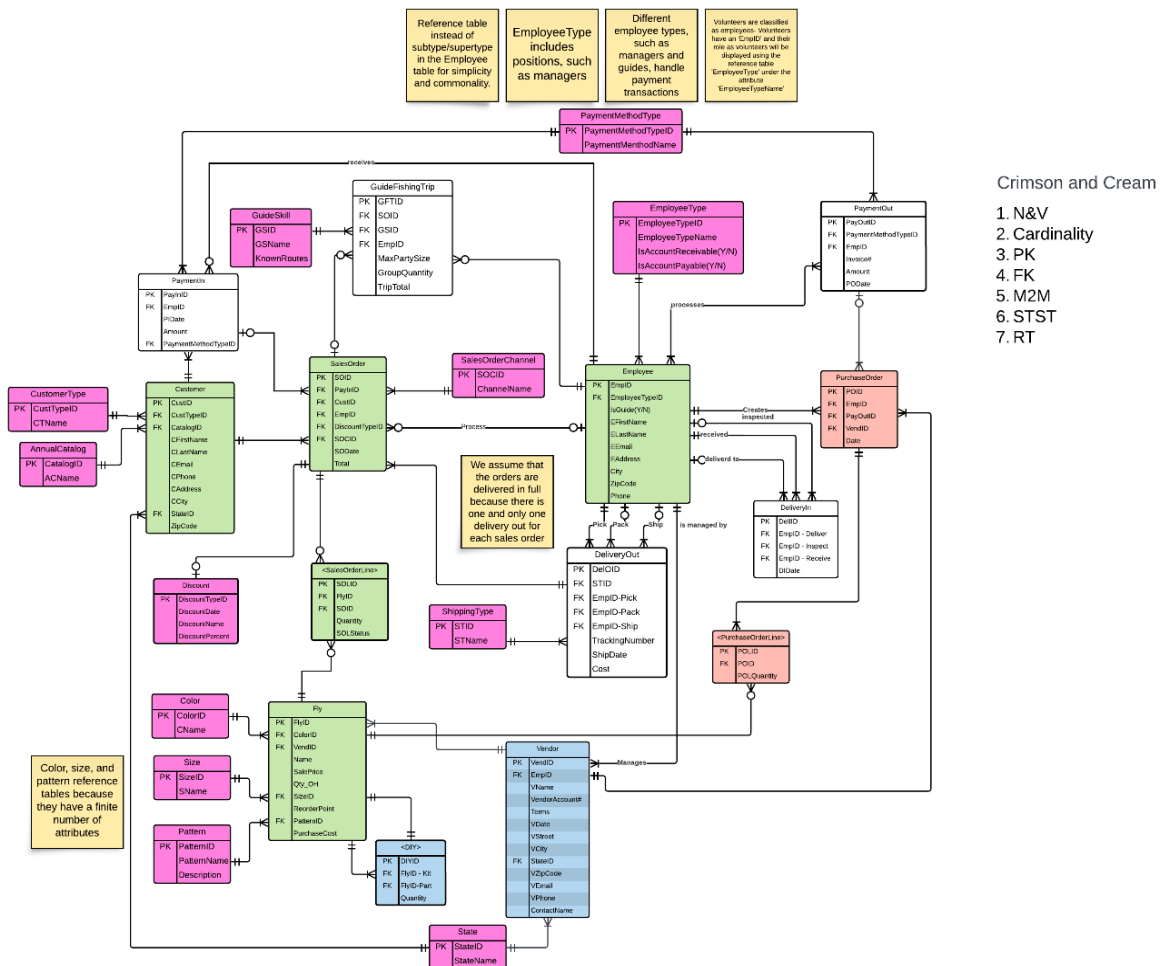
Vendor	Stellas Outdoor Shop			
ProductName	Cost	PatternID	SizeID	ColorID
SuperFly	\$8.99	2	101	212
Flyalicious	\$4.99	65	112	203
BigFishFlier	\$2.99	47	108	204
FlyingHigh	\$10.99	12	110	207

Vendor	BigRigFishing			
ProductName	Cost	PatternID	SizeID	ColorID
OhFlyGoodness	\$1.99	37	103	211
FlyFishinInTheDark	\$6.99	1	108	205
BackCountryFly	\$7.99	93	111	201
F-22	\$13.99	9	104	209

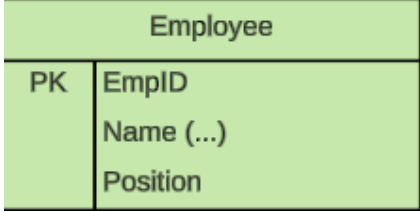
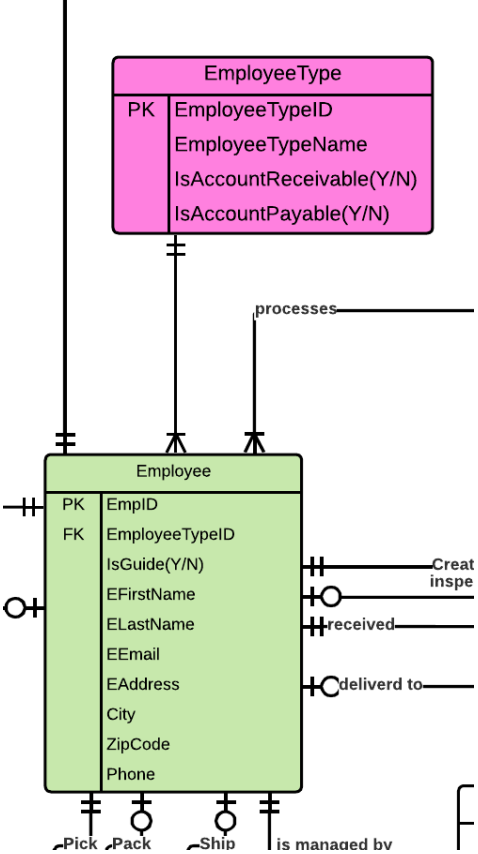
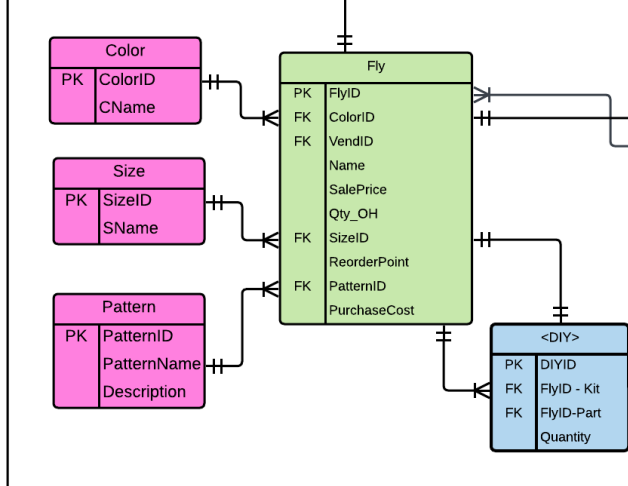
Vendor	BoomerSoonerFly			
ProductName	Cost	PatternID	SizeID	ColorID
ProgramFly	\$3.99	98	123	216
FlylerMurray	\$4.99	45	117	212
FlylySims	\$2.99	23	119	214
FlyLikeRoy	\$5.99	6	121	207

ERD Created

Link to lucid chart: https://lucid.app/lucidchart/ca4d8b1f-d9c4-4b73-9755-39d036481937/edit?invitationId=inv_d60bf05e-d15a-4da2-9357-1381a60ab70c&page=0_0#



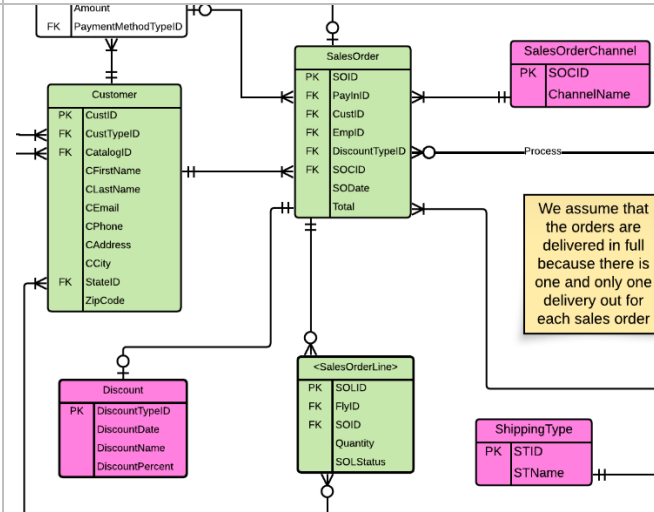
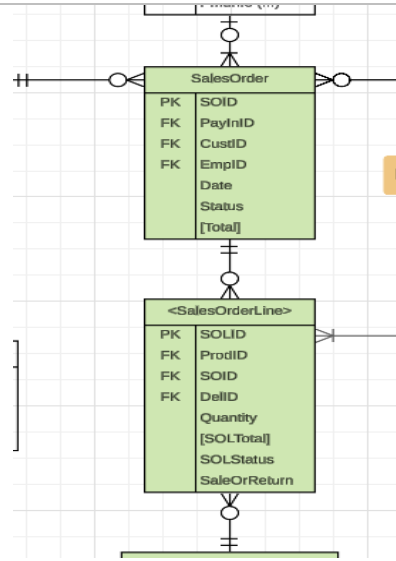
Changes made to generic ERDs

Change #	Original ERD	Updated ERD
For the 'Employee' entity, we made a relationship with 'PaymentIn', 'Vendor', 'PaymentOut', 'SalesOrder', 'GuideFishingTrip' and 'PurchaseOrder'. There are three different relationships with 'DeliveryIn' and three with 'DeliveryOut'. We also added a reference table titled, 'EmployeeType' because there are a finite number of types of employees. We did two attributes for first and last name, as well as a foreign key for 'EmployeeTypeID'.	 The original ERD for the 'Employee' entity is shown in a green box. It has a primary key 'EmpID' and attributes 'Name (...)' and 'Position'.	 The updated ERD for the 'Employee' entity is shown in a green box. It has a primary key 'EmpID' and attributes 'EmployeeTypeID' (foreign key), 'IsGuide(Y/N)', 'EFirstName', 'ELastName', 'EEmail', 'EAddress', 'City', 'ZipCode', and 'Phone'. It is connected to 'EmployeeType' (pink box) via a 1:1 relationship. It also has relationships with 'Pick', 'Pack', 'Ship', and 'is managed by'.
Changed table name from 'Product' to 'Fly' to specify the product. 'Fly' table has a relationship with 'DIY', 'Vendor', '<PurchaseOrderLine>', and '<SalesOrderLine>'. There are 'Color', 'Size', and 'Pattern' reference tables connecting to 'Fly' to display different materials used for a finished fly. Of course, there are foreign keys	 The original ERD for the 'Product' entity is shown in a green box. It has a primary key 'ProdID' and attributes 'Name', 'SalePrice', 'Qty_OH', 'Qty_Commit', '[Qty_Avail]', '[Qty_Backorder]', 'Location', and 'ReorderPoint'.	 The updated ERD for the 'Fly' entity is shown in a green box. It has a primary key 'FlyID' and attributes 'ColorID' (foreign key), 'VendID' (foreign key), 'Name', 'SalePrice', 'Qty_OH', 'SizeID' (foreign key), 'ReorderPoint', 'PatternID' (foreign key), and 'PurchaseCost'. It is connected to 'Color' (pink box), 'Size' (pink box), 'Pattern' (pink box), and '<DIY>' (blue box) via 1:1 relationships.

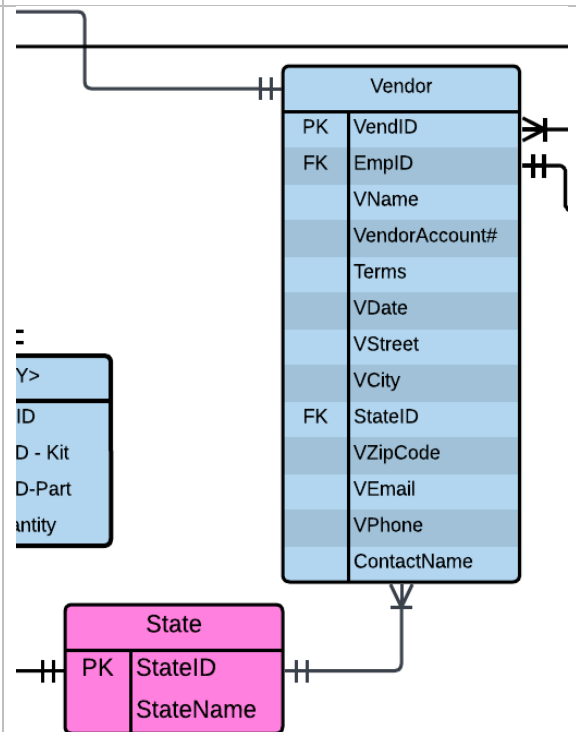
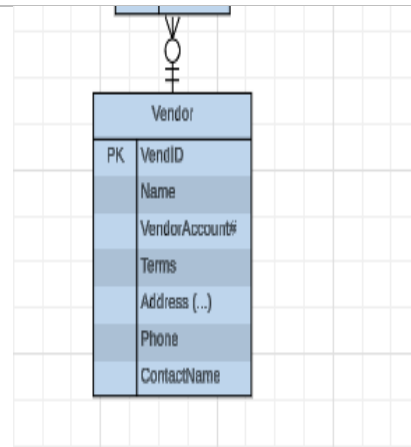
for each of these reference tables.

‘SalesOrder’ now has two reference tables titled ‘Discount’ and ‘SalesOrderChannel’ as well as foreign keys from those reference tables, ‘DiscountTypeID’ and ‘SOCID’. The ‘SalesOrder’ table has a relationship with ‘PaymentIn’, ‘GuideFishingTrip’, ‘DeliveryOut’, ‘Employee’, ‘<SalesOrderLine>’, and ‘Customer’.

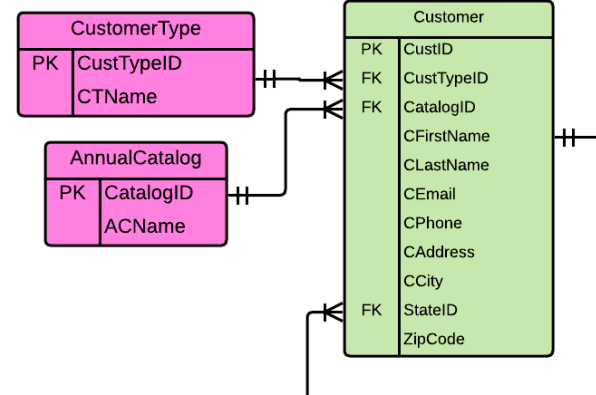
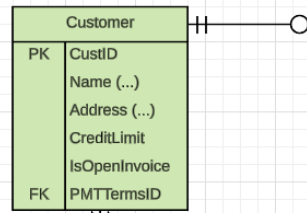
‘<SalesOrderLine>’ has relationships with ‘Fly’ and ‘SalesOrder’.



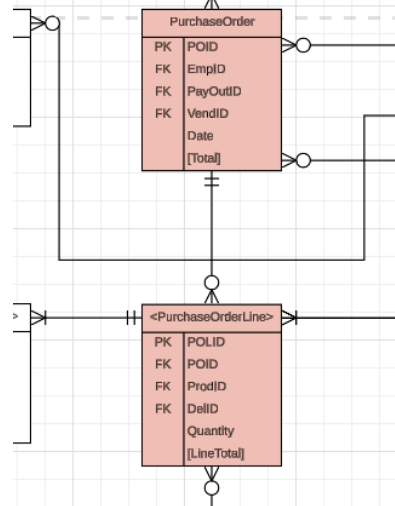
For ‘Vendor’, we attached one reference table, titled ‘State’ because there are a finite number of attributes. This of course means we added a ‘StateID’ foreign key. ‘Vendor’ has a relationship with the ‘PurchaseOrder’ tables for the different transactions. We added ‘Date’, ‘Terms’, and ‘VendorAccount#’ attributes. We also expanded the ‘Address’ attribute by exhausting all aspects of an address.



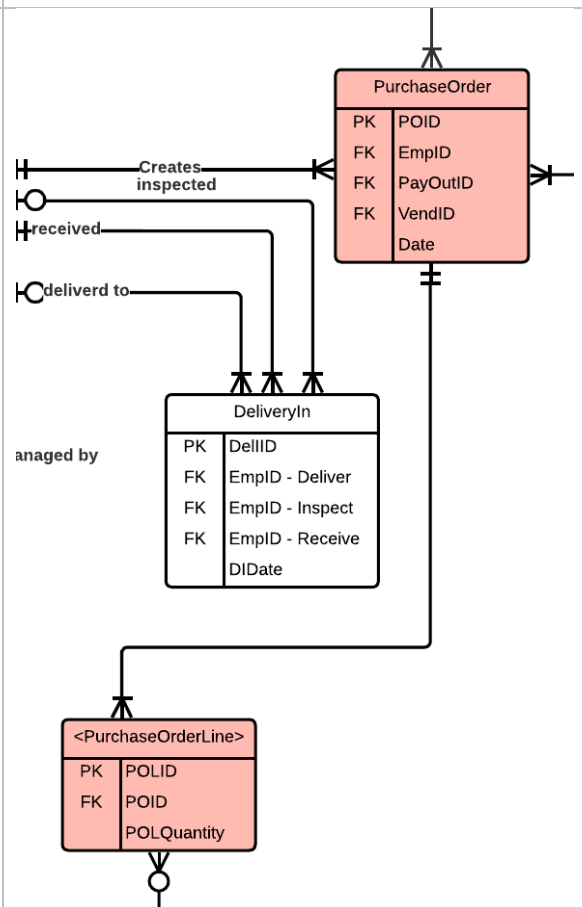
For 'Customer', we added two reference tables of 'AnnualCatalog' and 'CustomerType' because they all have a finite number of values, hence making them foreign keys in the customer table.



'PurchaseOrder' has a relationship with Employee, 'PaymentOut', 'Vendor', and '<PurchaseOrderLine>'.



'<PurchaseOrderLine>' has relationships with 'PurchaseOrder' and 'Fly' because of the variety of flies and the quantity of the product available.



Logical Design

Determining the constraints, connections, tables, and columns in compliance with the system requirements is the essence of logical design. Logical design uses normalization techniques to ensure data integrity and efficiency. The resultant logical schema serves as a design for the physical implementation of the database and controls data retrieval and storage. A comprehensive and intelligible depiction of the data model is the aim of logical design, as it forms the basis for creating the database system itself.

Normalization

Normalization is a process through which a database is made trustworthy, this also referred to as making sure it has “data integrity”. A database with data integrity is one in which you can trust the reports generated through it. Normalization contributes to overall database efficiency by eliminating data duplication and ensuring columns are broken down to their simplest form, also known as atomicity. To make sure that database tables are atomic and not redundant, normal forms are used. Normal forms indicate how far along the road to normalization a table currently is. There are four normal forms considered: zero normal form (0NF), first normal form (1NF), second normal form (2NF), and third normal form (3NF). The higher the form, the more normalized the table is. In this case, we make use of the third normal form.

Normalization of Data Provided by the Client

TVendor(VendorID, Vendor Name)

TProduct(ProductID, VendorID*, ProductName, Cost, PatternID, SizeID, ColorID)

Foreign Key VendorID references TVendor

Not Null

On Delete Restrict

Normalized Relations

TState(StateID, StateName)

TColor(ColorID, CName)

TSize(SizeID, SName)

TPattern(PatternID, PatternName, Description)

TAnnualCatalog(CatalogID, ACName)

TCustomerType(CustTypeID, CTName)

TEmployeeType(EmployeeTypeID, EmployeeTypeName, IsAccountReceivable(Y/N), IsAccountPayable(Y/N))

TPaymentMethodType(PaymentMethodTypeID, PaymentMethodName)

TShippingType(STID, STName)

TGuideSkill(GSID, GSName, KnownRoutes)

TSalesOrderChannel(SOCID, ChannelName)

TDiscount(DiscountTypeID, DiscountDate, DiscountName, DiscountPercent)

TEmployee(EmpID, EmployeeTypeID*, IsGuide(Y/N), EFirstName, ELastName, EAddress, City, ZipCode, Phone)

Foreign Key EmployeeTypeID references TEmployeeType

Not Null

On Delete Restrict

T<PurchaseOrderLine>(POLID, POID*, POLQuantity)

Foreign Key POID references TPurchaseOrder

Not Null

On Delete Restrict

TPaymentIn(PayInID, EmpID*, PIDate, Amount, PaymentMethodTypeID*)

Foreign Key EmpID references TEmployee

Not Null

On Delete Restrict

Foreign Key PaymentMethodID references TPaymentMethodType

Not Null

On Delete Restrict

T<SalesOrderLine>(SOLID, FlyID*, SOID*, Quantity, SOLStatus)

Foreign Key FlyID references TFly

Not Null

On Delete Restrict

Foreign Key SOID references TSalesOrder

Not Null

On Delete Restrict

T<DIY>(DIYID, FlyID-Kit*, FlyID-Part*, Quantity)

Foreign Key FlyID-Kit references TFly

Not Null

On Delete Restrict

Foreign Key FlyID-Part references TFly

Not Null

On Delete Restrict

TPaymentOut(PayOutID, PaymentMethodTypeID*, EmpID*, Invoice#, Amount, PODate)

Foreign Key PaymentMethodTypeID references TPaymentMethodType

Not Null

On Delete Restrict

TVendor(VendID, EmpID*, VName, VendorAccount#, Terms, Date, VStreet, VCity, StateID*, VZipCode, VPhone, ContactName)

Foreign Key FlyID references TFly

Not Null

On Delete Restrict

Foreign Key EmpID references TEmployee

Not Null

On Delete Restrict

Foreign Key StateID references TState
Not Null
On Delete Restrict

Foreign Key EmpID references TEmployee
Not Null
On Delete Restrict

TDeliveryIn(DelIID, EmpID-Deliver*, EmpID-Inspect*, EmpID-Receive *, DIDate)
Foreign Key EmpID- Deliver references TEmployee
Not Null
On Delete Restrict

Foreign Key EmpID-Inspect references TEmployee
Not Null
On Delete Restrict

Foreign Key EmpID-Receive references TEmployee
Not Null
On Delete Restrict

TDeliveryOut(DelOID, STID*, EmpID-Pick*, EmpID-Pack*, EmpID-Ship*, TrackingNumber,
ShipDate, Cost)
Foreign Key STID references TShippingType
Not Null
On Delete Restrict

Foreign Key EmpID-Pick references TEmployee
Not Null
On Delete Restrict

Foreign Key EmpID-Pack references TEmployee
Not Null
On Delete Restrict

Foreign Key EmpID-Ship references TEmployee
Not Null
On Delete Restrict

TGuideFishingTrip(GFTID, SOID*, GSID*, EmpID*, MaxPartySize, GroupQuantity,
TripTotal)
Foreign Key SOID references TSalesOrder
Not Null
On Delete Restrict

Foreign Key GSID references TGuideSkill
Not Null
On Delete Restrict

Foreign Key EmpID references TEmployee
Not Null
On Delete Restrict

TSalesOrder(SOID, PayInID*, CustID*, EmpID*, DiscountTypeID*, SODate, Total)
Foreign Key PayInID references TPaymentIn
Not Null
On Delete Restrict

Foreign Key CustID references TCustomer
Not Null
On Delete Restrict

Foreign Key EmpID references TEmployee
Not Null
On Delete Restrict

Foreign Key DiscountTypeID references TDiscountType
Not Null
On Delete Restrict

TPurchaseOrder(POID, EmpID*, PayOutID*, VendID*, Date)
Foreign Key EmpID references TEmployee
Not Null
On Delete Restrict

Foreign Key PayOutID references TPaymentOut
Not Null
On Delete Restrict

Foreign Key VendID references TVendor
Not Null
On Delete Restrict

TCustomer(CustID, CustTypeID*, CatalogID*, StateID*, CFirstName, CLastName, CEmail, CAddress, CCity, ZipCode, CreditLimit, IsOpenInvoive(Y/N))
Foreign Key CustTypeID references TCustomerType
Not Null
On Delete Restrict

Foreign Key CatalogID references TAnnualCatalog
Not Null
On Delete Restrict

Foreign Key StateID references TState
Not Null
On Delete Restrict

TOrderDetails(OrderID, CustID*, CFirstName, CLastName, GroupQuantity, PODate, Invoice#, TripTotal, FlySalePrice, Total, Status, DiscountPercent, PIDate)

Foreign Key CustID references TCustomer
Not Null
On Delete Restrict
TFly(FlyID, ColorID* VendID*, Name, SalePrice, Qty_OH, SizeID*, ReorderPoint, PatternID*, PurchaseCost)

Foreign Key ColorID references TColor
Not Null
On Delete Restrict

Foreign Key VendID references TVendor
Not Null
On Delete Restrict

Foreign Key SizeID references TSize
Not Null
On Delete Restrict

Foreign Key PatternID references TPattern
Not Null
On Delete Restrict

Differences between ERD and Normalized Relations

The difference between an ERD and a normalized relation is that an ERD is primarily conceptual, serving as a visual representation of the database's relationships, whereas normalization organizes the data in an ERD into more defined tables, with attributes broken down into their simplest form to reduce redundancy and improve data integrity. The formation of an ERD and a normalized relation occur at separate stages of the creation of a database, namely conceptual and logical design.

Normalized relations are useful in creating new databases and modifying existing ones. Normalized relations are easily implemented while creating a new database. Normalized relations in database modification enable the repair of flawed databases, they separate the content from the ERD structure, allowing for the development of an efficient, accurate database.

Referential Integrity

Referential integrity ensures that the relationships and data in a database are accurate. It guarantees that all references between the tables are accurate when correctly implemented. By linking a foreign key from one table to a primary key from another, this integrity constraint creates associations between different entity types. If the mandatory side is optional, the constraint may be empty. To ensure that there are always accurate links between tables, certain requirements must be fulfilled. They are crucial to a relation database because they create the rules that govern relationships, maintaining the usefulness and integrity of the database.

Relationships between entities would break down without referential integrity, leaving a disconnected database unable to link data correctly.

Physical Design and Implementation

Physical design refers to the process of organizing and arranging the physical storage structures of a database to optimize performance and efficiency. It involves decisions regarding how data is stored on disk, how indexes are structured, and how data is partitioned and distributed across storage devices or servers.

Implementation refers to the actual process of creating, configuring, and setting up a database system based on the specifications and design decisions made during the database design phase. It involves translating the logical database design into a physical database schema and configuring the database management system accordingly.

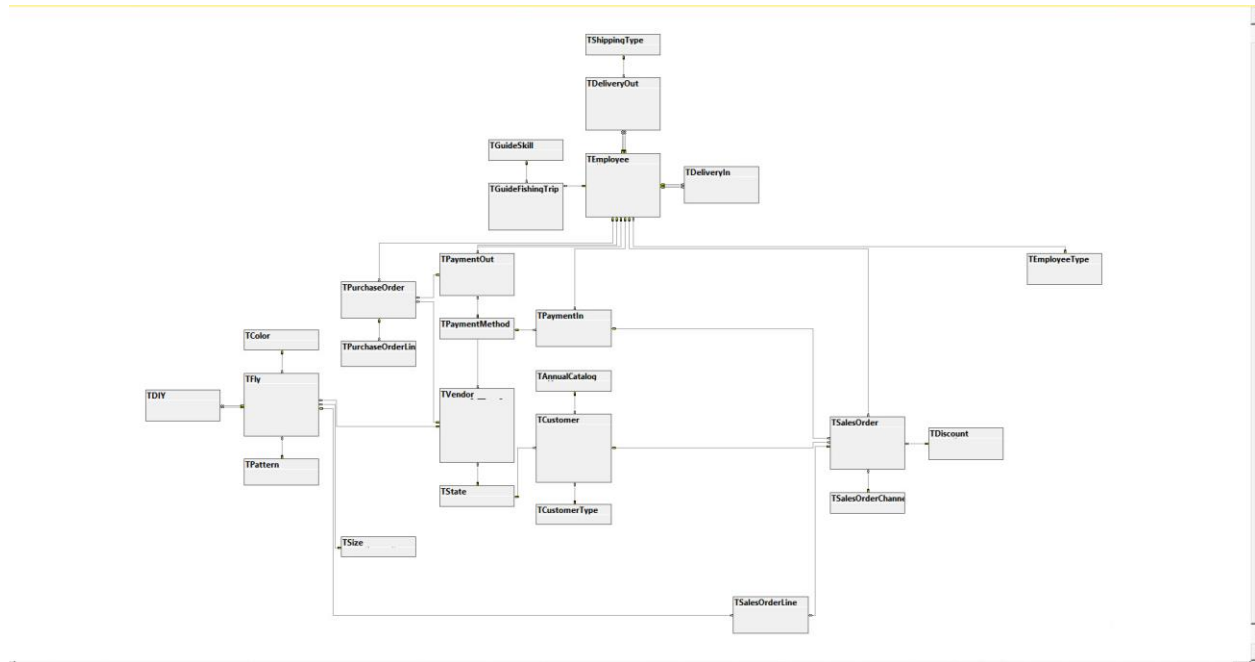
Data Dictionary

A Data Dictionary is a document that contains information regarding a record's data type in a table and its use-case within the database. It's important to know this information when creating the physical design of a database to improve accuracy when implementing new data in the database. An example of the necessity of a data dictionary is the physical design of the customer table for Elysian Fly Co. stating that 'CustID' is a primary key we would assign an integer for its datatype that would auto-increment each time a new customer was to be added into the table. Another example in the customer table is assigning the customer email address's data type as a varchar as an email address can contain special characters that are both integer and string.

Denormalization

By combining redundant data into a single table, denormalization improves read efficiency and speeds up database retrieval. Denormalization may also mean that a field's atomicity is being diminished. Since more and more tables are referenced, its main purpose is to shorten the time it takes for queries to execute. Denormalization could involve explicitly adding state information to the 'TCustomer' database by adding the 'StateID' from 'TState', for more details and explanations. Denormalization allows us to include the 'StateName' into 'TCustomer' straight from that statement. Denormalization simplifies queries containing customer state data and may improve performance.

Implemented Physical Design



Challenges Faced/Addressed During Implementation

We had several obstacles throughout the project's implementation phase that needed to be handled using proactive approaches and teamwork. Integrating several new ideas and strategies each with its own complexity and compatibility was a major task. To guarantee smooth functionality and integration, this required careful planning, rigorous testing, and iterative modifications. We also presented a difficulty necessitating efficient use of our resources and sensible work prioritization to fulfill project deadlines. We addressed this by putting in place thorough changes to our project, setting up more group meetings, and constant communication, to promote acceptance and lessen opposition. Simple errors like missing a comma or a word was misspelled happened frequently, but some of the bigger mistakes was trying to generate data and place it in the correct format/tables. Debugging errors and having to fix our SQL was also a constant problem within our group. Through hard work and great communication, we were able to get it all down and we are becoming more efficient with the system.

Strengths and Weaknesses Encountered During Implementation

Strengths

Catching Mistakes from ERD (Entity-Relationship Diagram): One of our strengths lies in catching mistakes from the ERD during implementation. This ensured that the database design was accurately translated into the actual tables and relationships, saving time and resources that would have otherwise been spent on correcting issues later.

Efficient Table Creation: Our implementation process was streamlined and efficient in creating tables. We utilized automated scripts for table creation, adhered to standardized naming

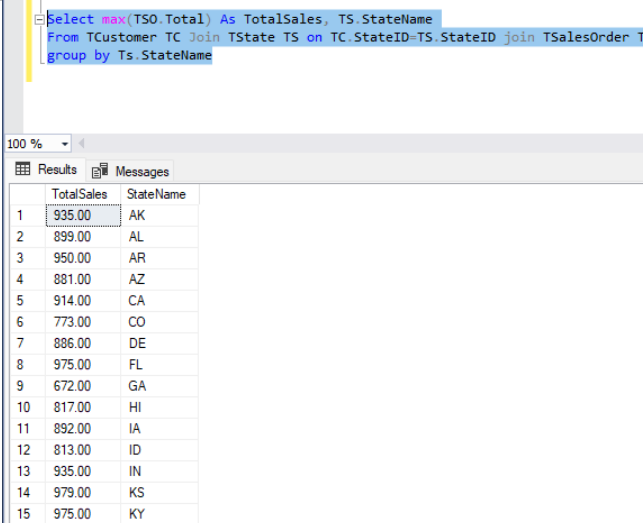
conventions, and provided proper documentation, all contributing to a smoother implementation phase.

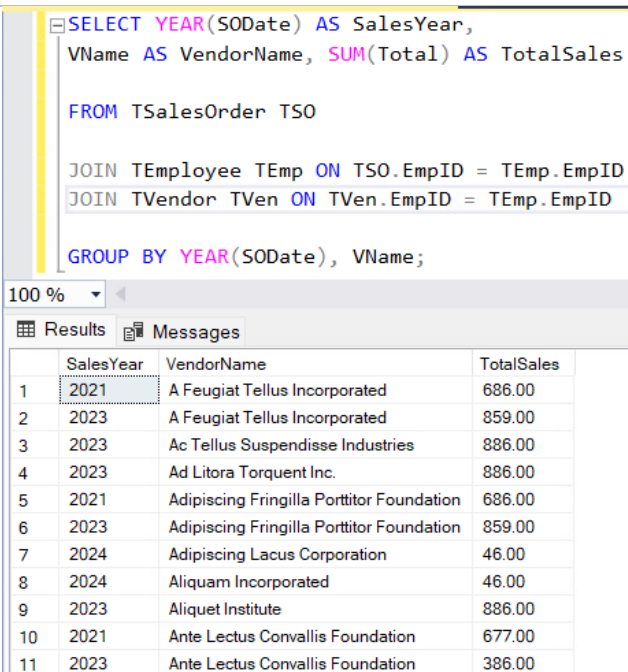
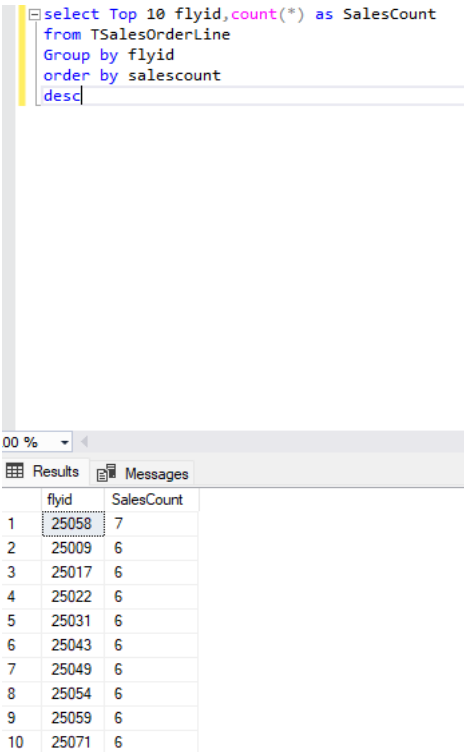
Weaknesses

Data Integrity Challenges: Maintaining data integrity, including accuracy, consistency, and reliability, was a significant challenge during implementation. This encompassed issues such as duplicate records, data validation errors, and inconsistencies in data entry.

Hardware Malfunctions: We were vulnerable to hardware malfunctions, including server failures, network interruptions, and storage device malfunctions, all of which disrupted the implementation process and lead to data loss or downtime.

Specific SQL Statements Requested

Query #	Question	SQL	Partial Output
1	Total sales in dollars by customer state per year	<pre>SELECT MAX(TSO.Total) AS TotalSales, TS.StateName FROM TCustomer TC JOIN TState TS ON TC.StateID=TS.StateID JOIN TSalesOrder TSO ON TSO.CustID=TC.CustID GROUP BY TS.StateName</pre> <i>Saved as a view</i>	

2	Total sales in dollars by vendor per year	<pre>SELECT YEAR(SODate) AS SalesYear, VName AS VendorName, SUM(Total) AS TotalSales FROM TSalesOrder TSO JOIN TEmployee TTemp ON TSO.EmpID = TTemp.EmpID JOIN TVendor TVen ON TVen.EmpID = TTemp.EmpID GROUP BY YEAR(SODate), VName;</pre> <i>Saved as a view</i>	
3	The ten highest selling a) patterns, b) sizes c) pattern-size color combinations	(a) Highest selling patterns -- Top 10 patterns <pre>SELECT Top 10 FlyID, COUNT(*) as SalesCount FROM TSalesOrderLine GROUP BY FlyID ORDER BY SalesCount DESC</pre> <i>Saved as a procedure</i> (b) Highest selling sizes <pre>SELECT Top 10 TS.SizeID, COUNT(*) AS SalesCount FROM TSalesOrderLine TSO Join TFly TF on TSO.FlyID = TF.FlyID join TSize TS on TF.SizeID = TS.SizeID GROUP BY TS.SizeID</pre>	

ORDER BY SalesCount DESC

Saved as a procedure

(c) Highest selling pattern-size-color combinations

-- Top 10 pattern-size-color combinations

```
SELECT TF.FlyID, ColorID,
       TS.SizeID, COUNT(*) AS
       SalesCount
```

```
FROM TSalesOrderLine TSO
Join TFly TF on TSO.FlyID =
TF.FlyID join TSize TS on
TF.SizeID = TS.SizeID
```

```
GROUP BY TF.FlyID, ColorID,
          TS.SizeID
```

ORDER BY SalesCount DESC

Saved as a procedure

```
SELECT TS.SizeID, COUNT(*) AS SalesCount

FROM TSalesOrderLine TSO Join TFly TF on TSO.FlyID = TF.FlyID
join TSize TS on TF.SizeID = TS.SizeID

GROUP BY TS.SizeID

ORDER BY SalesCount DESC;
```

100 %

Results Messages

	SizeID	SalesCount
1	2002	50
2	2008	47
3	2007	42
4	2006	35
5	2001	34
6	2004	33
7	2005	25
8	2003	24
9	2009	22

```
SELECT Top 10 TS.SizeID, COUNT(*) AS SalesCount

FROM TSalesOrderLine TSO Join TFly TF on TSO.FlyID = TF.FlyID join TSize TS on TF.SizeID

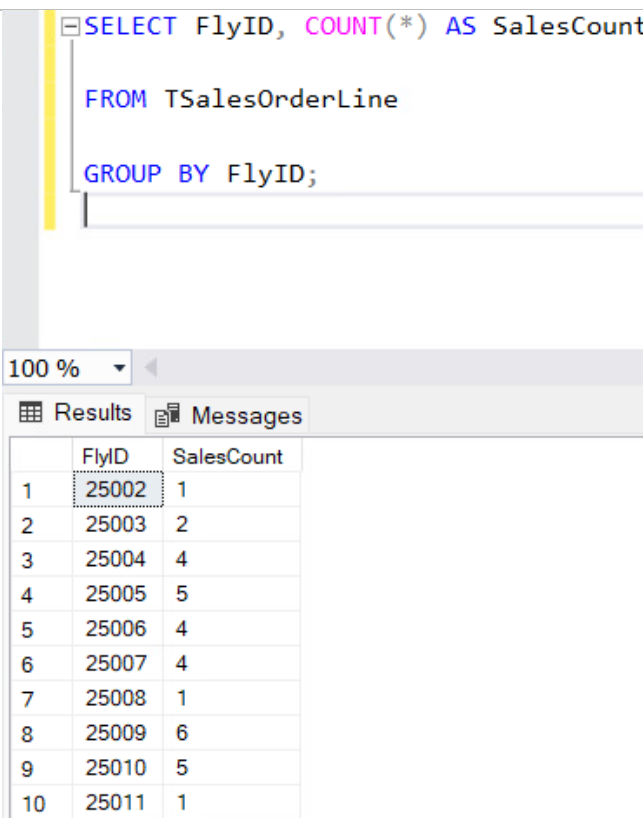
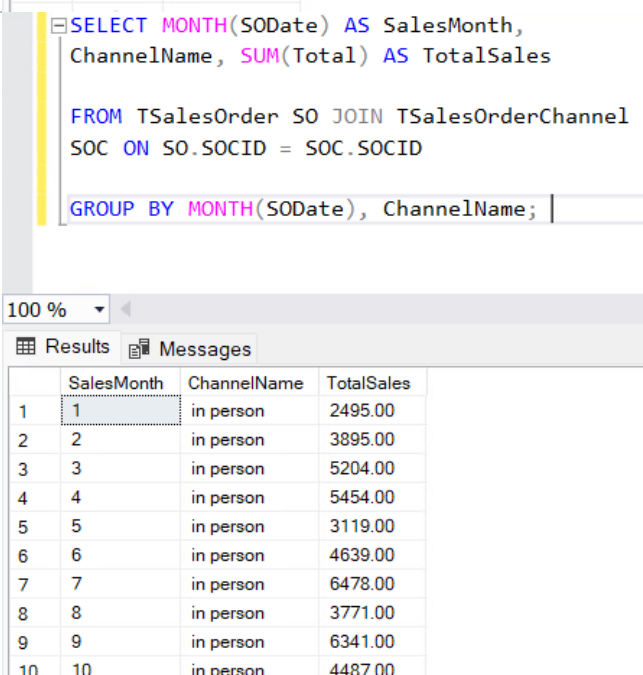
GROUP BY TS.SizeID

ORDER BY SalesCount DESC
```

%

Results Messages

	SizeID	SalesCount
	2002	50
	2008	47
	2007	42
	2006	35
	2001	34
	2004	33
	2005	25
	2003	24
	2009	22

4	Number of times each product (fly) was sold -Also want to see least sold to discontinue them	<pre>SELECT FlyID, COUNT(*) AS SalesCount FROM TSalesOrderLine GROUP BY FlyID;</pre> <i>Saved as a view</i>	 <pre>SELECT FlyID, COUNT(*) AS SalesCount FROM TSalesOrderLine GROUP BY FlyID;</pre> <table><thead><tr><th></th><th>FlyID</th><th>SalesCount</th></tr></thead><tbody><tr><td>1</td><td>25002</td><td>1</td></tr><tr><td>2</td><td>25003</td><td>2</td></tr><tr><td>3</td><td>25004</td><td>4</td></tr><tr><td>4</td><td>25005</td><td>5</td></tr><tr><td>5</td><td>25006</td><td>4</td></tr><tr><td>6</td><td>25007</td><td>4</td></tr><tr><td>7</td><td>25008</td><td>1</td></tr><tr><td>8</td><td>25009</td><td>6</td></tr><tr><td>9</td><td>25010</td><td>5</td></tr><tr><td>10</td><td>25011</td><td>1</td></tr></tbody></table>		FlyID	SalesCount	1	25002	1	2	25003	2	3	25004	4	4	25005	5	5	25006	4	6	25007	4	7	25008	1	8	25009	6	9	25010	5	10	25011	1											
	FlyID	SalesCount																																													
1	25002	1																																													
2	25003	2																																													
3	25004	4																																													
4	25005	5																																													
5	25006	4																																													
6	25007	4																																													
7	25008	1																																													
8	25009	6																																													
9	25010	5																																													
10	25011	1																																													
5	Total sales in dollars for each channel per month	<pre>SELECT MONTH(SODate) AS SalesMonth, ChannelName, SUM(Total) AS TotalSales FROM TSalesOrder SO JOIN TSalesOrderChannel SOC ON SO.SOCID = SOC.SOCID GROUP BY MONTH(SODate), ChannelName;</pre> <i>Saved as a view</i>	 <pre>SELECT MONTH(SODate) AS SalesMonth, ChannelName, SUM(Total) AS TotalSales FROM TSalesOrder SO JOIN TSalesOrderChannel SOC ON SO.SOCID = SOC.SOCID GROUP BY MONTH(SODate), ChannelName;</pre> <table><thead><tr><th></th><th>SalesMonth</th><th>ChannelName</th><th>TotalSales</th></tr></thead><tbody><tr><td>1</td><td>1</td><td>in person</td><td>2495.00</td></tr><tr><td>2</td><td>2</td><td>in person</td><td>3895.00</td></tr><tr><td>3</td><td>3</td><td>in person</td><td>5204.00</td></tr><tr><td>4</td><td>4</td><td>in person</td><td>5454.00</td></tr><tr><td>5</td><td>5</td><td>in person</td><td>3119.00</td></tr><tr><td>6</td><td>6</td><td>in person</td><td>4639.00</td></tr><tr><td>7</td><td>7</td><td>in person</td><td>6478.00</td></tr><tr><td>8</td><td>8</td><td>in person</td><td>3771.00</td></tr><tr><td>9</td><td>9</td><td>in person</td><td>6341.00</td></tr><tr><td>10</td><td>10</td><td>in person</td><td>4487.00</td></tr></tbody></table>		SalesMonth	ChannelName	TotalSales	1	1	in person	2495.00	2	2	in person	3895.00	3	3	in person	5204.00	4	4	in person	5454.00	5	5	in person	3119.00	6	6	in person	4639.00	7	7	in person	6478.00	8	8	in person	3771.00	9	9	in person	6341.00	10	10	in person	4487.00
	SalesMonth	ChannelName	TotalSales																																												
1	1	in person	2495.00																																												
2	2	in person	3895.00																																												
3	3	in person	5204.00																																												
4	4	in person	5454.00																																												
5	5	in person	3119.00																																												
6	6	in person	4639.00																																												
7	7	in person	6478.00																																												
8	8	in person	3771.00																																												
9	9	in person	6341.00																																												
10	10	in person	4487.00																																												

6	The 10% of products that have the highest margin	SELECT TOP 10 PERCENT FlyID, Name AS ProductName, (SalePrice - PurchaseCost) AS Margin FROM TFly ORDER BY Margin DESC; Saved as a view	SELECT TOP 10 PERCENT FlyID, Name AS ProductName, (SalePrice - PurchaseCost) AS Margin FROM TFly ORDER BY Margin DESC; 100 % Results Messages <table><tr><th></th><th>FlyID</th><th>ProductName</th><th>Margin</th></tr><tr><td>1</td><td>25102</td><td>Summer Kit</td><td>45.00</td></tr><tr><td>2</td><td>25105</td><td>Summer Kit Parts</td><td>40.00</td></tr><tr><td>3</td><td>25032</td><td>Brassie</td><td>9.00</td></tr><tr><td>4</td><td>25069</td><td>Brassie</td><td>9.00</td></tr><tr><td>5</td><td>25055</td><td>Egg-Sucking Leech</td><td>8.00</td></tr><tr><td>6</td><td>25039</td><td>Woolly Worm</td><td>8.00</td></tr></table>		FlyID	ProductName	Margin	1	25102	Summer Kit	45.00	2	25105	Summer Kit Parts	40.00	3	25032	Brassie	9.00	4	25069	Brassie	9.00	5	25055	Egg-Sucking Leech	8.00	6	25039	Woolly Worm	8.00																												
	FlyID	ProductName	Margin																																																								
1	25102	Summer Kit	45.00																																																								
2	25105	Summer Kit Parts	40.00																																																								
3	25032	Brassie	9.00																																																								
4	25069	Brassie	9.00																																																								
5	25055	Egg-Sucking Leech	8.00																																																								
6	25039	Woolly Worm	8.00																																																								
7	The ten most popular (units sold) DIY fly-tying materials	SELECT FlyID, SUM(Quantity) AS TotalUnitsSold FROM TSalesOrderLine GROUP BY FlyID ORDER BY TotalUnitsSold DESC Saved as a procedure	SELECT FlyID, SUM(Quantity) AS TotalUnitsSold FROM TSalesOrderLine GROUP BY FlyID ORDER BY TotalUnitsSold DESC 100 % Results Messages <table><tr><th></th><th>FlyID</th><th>TotalUnitsSold</th></tr><tr><td>1</td><td>25054</td><td>443</td></tr><tr><td>2</td><td>25022</td><td>406</td></tr><tr><td>3</td><td>25058</td><td>401</td></tr><tr><td>4</td><td>25091</td><td>380</td></tr><tr><td>5</td><td>25095</td><td>347</td></tr><tr><td>6</td><td>25059</td><td>315</td></tr><tr><td>7</td><td>25009</td><td>315</td></tr><tr><td>8</td><td>25093</td><td>313</td></tr><tr><td>9</td><td>25017</td><td>310</td></tr><tr><td>10</td><td>25005</td><td>300</td></tr></table>		FlyID	TotalUnitsSold	1	25054	443	2	25022	406	3	25058	401	4	25091	380	5	25095	347	6	25059	315	7	25009	315	8	25093	313	9	25017	310	10	25005	300																							
	FlyID	TotalUnitsSold																																																									
1	25054	443																																																									
2	25022	406																																																									
3	25058	401																																																									
4	25091	380																																																									
5	25095	347																																																									
6	25059	315																																																									
7	25009	315																																																									
8	25093	313																																																									
9	25017	310																																																									
10	25005	300																																																									
8	The number of distinct products managed by each vendor manager	SELECT E.EmpID AS VendorManagerID, E.EFirstName AS VendorManagerFirstName, E.ELastName AS VendorManagerLastName, COUNT(DISTINCT F.FlyID) AS TotalDistinctProducts FROM TEmployee E JOIN TVendor V ON E.EmpID = V.VendID JOIN Fly F ON V.VendID = F.VendID	E.EmpID AS VendorManagerID, E.EFirstName AS VendorManagerFirstName, E.ELastName AS VendorManagerLastName, COUNT(DISTINCT F.FlyID) AS TotalDistinctProducts FROM TEmployee E JOIN TVendor V ON E.EmpID = V.VendID JOIN TFLy F ON V.VendID = F.VendID GROUP BY E.EmpID, E.EFirstName, E.ELastName; % Results Messages <table><tr><th>VendorManagerID</th><th>VendorManagerFirstName</th><th>VendorManagerLastName</th><th>TotalDistinctProducts</th></tr><tr><td>11005</td><td>Michael</td><td>Hernandez</td><td>15</td></tr><tr><td>11012</td><td>Amanda</td><td>Thompson</td><td>21</td></tr><tr><td>11013</td><td>Michael</td><td>Hernandez</td><td>3</td></tr><tr><td>11028</td><td>David</td><td>Thompson</td><td>36</td></tr><tr><td>11040</td><td>Daniel</td><td>Lopez</td><td>12</td></tr><tr><td>11042</td><td>Christopher</td><td>Lee</td><td>2</td></tr><tr><td>11044</td><td>Michael</td><td>Hernandez</td><td>2</td></tr><tr><td>11052</td><td>Ashley</td><td>Taylor</td><td>9</td></tr><tr><td>11053</td><td>Christopher</td><td>King</td><td>1</td></tr><tr><td>11057</td><td>Matthew</td><td>Wilson</td><td>8</td></tr><tr><td>11070</td><td>Emily</td><td>Thompson</td><td>26</td></tr><tr><td>11076</td><td>Sarah</td><td>Johnson</td><td>12</td></tr><tr><td>11077</td><td>David</td><td>Davis</td><td>2</td></tr></table>	VendorManagerID	VendorManagerFirstName	VendorManagerLastName	TotalDistinctProducts	11005	Michael	Hernandez	15	11012	Amanda	Thompson	21	11013	Michael	Hernandez	3	11028	David	Thompson	36	11040	Daniel	Lopez	12	11042	Christopher	Lee	2	11044	Michael	Hernandez	2	11052	Ashley	Taylor	9	11053	Christopher	King	1	11057	Matthew	Wilson	8	11070	Emily	Thompson	26	11076	Sarah	Johnson	12	11077	David	Davis	2
VendorManagerID	VendorManagerFirstName	VendorManagerLastName	TotalDistinctProducts																																																								
11005	Michael	Hernandez	15																																																								
11012	Amanda	Thompson	21																																																								
11013	Michael	Hernandez	3																																																								
11028	David	Thompson	36																																																								
11040	Daniel	Lopez	12																																																								
11042	Christopher	Lee	2																																																								
11044	Michael	Hernandez	2																																																								
11052	Ashley	Taylor	9																																																								
11053	Christopher	King	1																																																								
11057	Matthew	Wilson	8																																																								
11070	Emily	Thompson	26																																																								
11076	Sarah	Johnson	12																																																								
11077	David	Davis	2																																																								

		GROUP BY E.EmpID, E.EFirstName, E.ELastName; Saved as a view																																																																																												
9	The upcoming, scheduled guided trips	SELECT TEmployee.EFirstName AS GuideFirstName, TEmployee.ELastName AS GuideLastName, TSalesOrder.SODate AS TripDate, TGuideFishingTrip.MaxPartySize AS PartySize, TCustomer.CFirstName AS CustomerFirstName, TCustomer.CLastName AS CustomerLastName FROM TSalesOrder JOIN TGuideFishingTrip ON TSalesOrder.SOID = TGuideFishingTrip.SOID JOIN TEmployee ON TGuideFishingTrip.EmpID = TEmployee.EmpID JOIN TCustomer ON TSalesOrder.CustID = TCustomer.CustID WHERE TSalesOrder.SODate <= GetDate();	<pre>--SELECT TEmployee.EFirstName AS GuideFirstName, TEmployee.ELastName AS GuideLastName TSalesOrder.SODate AS TripDate, TGuideFishingTrip.MaxPartySize AS PartySize, TCustomer.CFirstName AS CustomerFirstName, TCustomer.CLastName AS CustomerLastName FROM TSalesOrder JOIN TGuideFishingTrip ON TSalesOrder.SOID = TGuideFishingTrip.SOID JOIN TEmployee ON TGuideFishingTrip.EmpID = TEmployee.EmpID JOIN TCustomer ON TSalesOrder.CustID = TCustomer.CustID WHERE TSalesOrder.SODate <= GetDate();</pre> 100 % Results Messages <table><tr><th></th><th>GuideFirstName</th><th>GuideLastName</th><th>TripDate</th><th>PartySize</th><th>CustomerFirstName</th><th>CustomerLastName</th></tr><tr><td>1</td><td>John</td><td>Rodriguez</td><td>2021-02-20</td><td>17</td><td>Eaton</td><td>Wall</td></tr><tr><td>2</td><td>Jennifer</td><td>Martinez</td><td>2023-05-29</td><td>17</td><td>Cyrus</td><td>Levine</td></tr><tr><td>3</td><td>David</td><td>Martinez</td><td>2023-12-12</td><td>13</td><td>Asher</td><td>Berg</td></tr><tr><td>4</td><td>Sarah</td><td>Hernandez</td><td>2021-05-09</td><td>12</td><td>Kessie</td><td>Baxter</td></tr><tr><td>5</td><td>David</td><td>Martinez</td><td>2024-03-15</td><td>19</td><td>Carter</td><td>Leon</td></tr><tr><td>6</td><td>Robert</td><td>Thomas</td><td>2022-03-01</td><td>17</td><td>Abigail</td><td>Pearson</td></tr><tr><td>7</td><td>Sarah</td><td>Hernandez</td><td>2022-04-21</td><td>17</td><td>Ivor</td><td>Gray</td></tr><tr><td>8</td><td>Sarah</td><td>Wilson</td><td>2021-01-16</td><td>19</td><td>Clark</td><td>Perry</td></tr><tr><td>9</td><td>Jennifer</td><td>Martinez</td><td>2022-11-22</td><td>12</td><td>Clark</td><td>Perry</td></tr><tr><td>10</td><td>Samantha</td><td>Gonzalez</td><td>2022-04-23</td><td>11</td><td>Felix</td><td>Mccoy</td></tr><tr><td>11</td><td>Christopher</td><td>Taylor</td><td>2022-07-12</td><td>14</td><td>Ezekiel</td><td>Cleveland</td></tr><tr><td>12</td><td>Robert</td><td>Thomas</td><td>2023-09-26</td><td>17</td><td>Lucius</td><td>Cash</td></tr></table>		GuideFirstName	GuideLastName	TripDate	PartySize	CustomerFirstName	CustomerLastName	1	John	Rodriguez	2021-02-20	17	Eaton	Wall	2	Jennifer	Martinez	2023-05-29	17	Cyrus	Levine	3	David	Martinez	2023-12-12	13	Asher	Berg	4	Sarah	Hernandez	2021-05-09	12	Kessie	Baxter	5	David	Martinez	2024-03-15	19	Carter	Leon	6	Robert	Thomas	2022-03-01	17	Abigail	Pearson	7	Sarah	Hernandez	2022-04-21	17	Ivor	Gray	8	Sarah	Wilson	2021-01-16	19	Clark	Perry	9	Jennifer	Martinez	2022-11-22	12	Clark	Perry	10	Samantha	Gonzalez	2022-04-23	11	Felix	Mccoy	11	Christopher	Taylor	2022-07-12	14	Ezekiel	Cleveland	12	Robert	Thomas	2023-09-26	17	Lucius	Cash
	GuideFirstName	GuideLastName	TripDate	PartySize	CustomerFirstName	CustomerLastName																																																																																								
1	John	Rodriguez	2021-02-20	17	Eaton	Wall																																																																																								
2	Jennifer	Martinez	2023-05-29	17	Cyrus	Levine																																																																																								
3	David	Martinez	2023-12-12	13	Asher	Berg																																																																																								
4	Sarah	Hernandez	2021-05-09	12	Kessie	Baxter																																																																																								
5	David	Martinez	2024-03-15	19	Carter	Leon																																																																																								
6	Robert	Thomas	2022-03-01	17	Abigail	Pearson																																																																																								
7	Sarah	Hernandez	2022-04-21	17	Ivor	Gray																																																																																								
8	Sarah	Wilson	2021-01-16	19	Clark	Perry																																																																																								
9	Jennifer	Martinez	2022-11-22	12	Clark	Perry																																																																																								
10	Samantha	Gonzalez	2022-04-23	11	Felix	Mccoy																																																																																								
11	Christopher	Taylor	2022-07-12	14	Ezekiel	Cleveland																																																																																								
12	Robert	Thomas	2023-09-26	17	Lucius	Cash																																																																																								
10	Number of trips and number of customers taken on fishing trips by each guide in the past 6 months	SELECT E.EmpID AS GuideID, E.EFirstName AS GuideFirstName, E.ELastName AS GuideLastName, COUNT(DISTINCT GFTID) AS TotalTrips, COUNT(DISTINCT CustID) AS TotalCustomers FROM TEmployee E JOIN TGuideFishingTrip G ON E.EmpID = G.EmpID join TSalesOrder SO on G.SOID = SO.SOID	<pre>--SELECT E.EmpID AS GuideID, E.EFirstName AS GuideFirstName, E.ELastName AS GuideLastName, COUNT(DISTINCT GFTID) AS TotalTrips COUNT(DISTINCT CustID) AS TotalCustomers FROM TEmployee E JOIN TGuideFishingTrip G ON E.EmpID = G.EmpID join TSalesOrder SO on G.SOID = SO.SOID WHERE SO.SODate >= DATEADD(MONTH, -6, GETDATE())</pre> 100 % Results Messages <table><tr><th></th><th>GuideID</th><th>GuideFirstName</th><th>GuideLastName</th><th>TotalTrips</th><th>TotalCustomers</th></tr><tr><td>1</td><td>11091</td><td>William</td><td>Flores</td><td>1</td><td>1</td></tr><tr><td>2</td><td>11098</td><td>Sarah</td><td>Hernandez</td><td>1</td><td>1</td></tr><tr><td>3</td><td>11100</td><td>David</td><td>Martinez</td><td>3</td><td>3</td></tr></table>		GuideID	GuideFirstName	GuideLastName	TotalTrips	TotalCustomers	1	11091	William	Flores	1	1	2	11098	Sarah	Hernandez	1	1	3	11100	David	Martinez	3	3																																																																			
	GuideID	GuideFirstName	GuideLastName	TotalTrips	TotalCustomers																																																																																									
1	11091	William	Flores	1	1																																																																																									
2	11098	Sarah	Hernandez	1	1																																																																																									
3	11100	David	Martinez	3	3																																																																																									

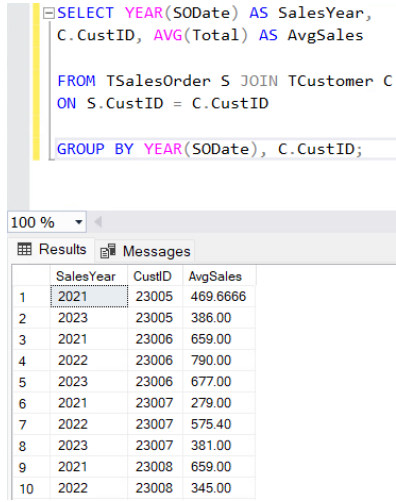
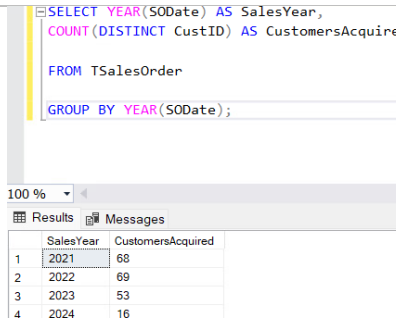
		WHERE SO.SODate >= DATEADD(MONTH, -6, GETDATE()) GROUP BY E.EmpID, E.EFirstName, E.ELastName; <i>Saved as a view</i>																																																																																	
11	Names and email addresses of all customers who made purchases in a given month	SELECT C.CFirstName AS CustomerFirstName, C.CLastName AS CustomerLastName, C.CEmail AS CustomerEmail FROM TCustomer C JOIN TSalesOrder S ON C.CustID = S.CustID WHERE MONTH(S.SODate) = <month>; <i>Saved as a view</i>	<pre>use UOKA0941 go SELECT TCustomer.CFirstName, TCustomer.CLastName, TCustomer.CEmail FROM TSalesOrder JOIN TCustomer ON TSalesOrder.CustID = TCustomer.CustID WHERE SODate = (TSalesOrder.SODate);</pre> 100 % Results Messages <table><thead><tr><th></th><th>CFirstName</th><th>CLastName</th><th>CEmail</th></tr></thead><tbody><tr><td>1</td><td>Nehru</td><td>Patterson</td><td>aliquet.phasellus.fermentum@yahoo.ca</td></tr><tr><td>2</td><td>Nehru</td><td>Patterson</td><td>aliquet.phasellus.fermentum@yahoo.ca</td></tr><tr><td>3</td><td>Nehru</td><td>Patterson</td><td>aliquet.phasellus.fermentum@yahoo.ca</td></tr><tr><td>4</td><td>Nehru</td><td>Patterson</td><td>aliquet.phasellus.fermentum@yahoo.ca</td></tr><tr><td>5</td><td>Abigail</td><td>Pearson</td><td>at.pede.cras@google.net</td></tr><tr><td>6</td><td>Abigail</td><td>Pearson</td><td>at.pede.cras@google.net</td></tr><tr><td>7</td><td>Abigail</td><td>Pearson</td><td>at.pede.cras@google.net</td></tr><tr><td>8</td><td>Abigail</td><td>Pearson</td><td>at.pede.cras@google.net</td></tr><tr><td>9</td><td>Zane</td><td>Hoover</td><td>quis.diam.pellentesque@icloud.edu</td></tr><tr><td>10</td><td>Zane</td><td>Hoover</td><td>quis.diam.pellentesque@icloud.edu</td></tr><tr><td>11</td><td>Zane</td><td>Hoover</td><td>quis.diam.pellentesque@icloud.edu</td></tr><tr><td>12</td><td>Zane</td><td>Hoover</td><td>quis.diam.pellentesque@icloud.edu</td></tr></tbody></table>		CFirstName	CLastName	CEmail	1	Nehru	Patterson	aliquet.phasellus.fermentum@yahoo.ca	2	Nehru	Patterson	aliquet.phasellus.fermentum@yahoo.ca	3	Nehru	Patterson	aliquet.phasellus.fermentum@yahoo.ca	4	Nehru	Patterson	aliquet.phasellus.fermentum@yahoo.ca	5	Abigail	Pearson	at.pede.cras@google.net	6	Abigail	Pearson	at.pede.cras@google.net	7	Abigail	Pearson	at.pede.cras@google.net	8	Abigail	Pearson	at.pede.cras@google.net	9	Zane	Hoover	quis.diam.pellentesque@icloud.edu	10	Zane	Hoover	quis.diam.pellentesque@icloud.edu	11	Zane	Hoover	quis.diam.pellentesque@icloud.edu	12	Zane	Hoover	quis.diam.pellentesque@icloud.edu																												
	CFirstName	CLastName	CEmail																																																																																
1	Nehru	Patterson	aliquet.phasellus.fermentum@yahoo.ca																																																																																
2	Nehru	Patterson	aliquet.phasellus.fermentum@yahoo.ca																																																																																
3	Nehru	Patterson	aliquet.phasellus.fermentum@yahoo.ca																																																																																
4	Nehru	Patterson	aliquet.phasellus.fermentum@yahoo.ca																																																																																
5	Abigail	Pearson	at.pede.cras@google.net																																																																																
6	Abigail	Pearson	at.pede.cras@google.net																																																																																
7	Abigail	Pearson	at.pede.cras@google.net																																																																																
8	Abigail	Pearson	at.pede.cras@google.net																																																																																
9	Zane	Hoover	quis.diam.pellentesque@icloud.edu																																																																																
10	Zane	Hoover	quis.diam.pellentesque@icloud.edu																																																																																
11	Zane	Hoover	quis.diam.pellentesque@icloud.edu																																																																																
12	Zane	Hoover	quis.diam.pellentesque@icloud.edu																																																																																
12	Number of times used, and dollars spent on each vendor and each shipping type by vendor	SELECT V.VendID, V.VName AS VendorName, ST.STName AS ShippingType, COUNT(DISTINCT S.SOID) AS TotalOrders, SUM(S.Total) AS TotalSpent FROM TVendor V JOIN TPurchaseOrder P ON V.VendID = P.VendID JOIN TEmployee E on P.EmpID=E.EmpID JOIN TSalesOrder S ON S.SOID = S.SOID JOIN TDeliveryOut D ON E.EmpID = D.EmpIDShip ON E.EmpID = D.EmpIDShip JOIN TShippingType ST ON D.STID = ST.STID GROUP BY V.VendID, V.VName, ST.STName; <i>Saved as a view</i>	<pre>use uoka0941 go SELECT V.VendID, V.VName AS VendorName, ST.STName AS ShippingType COUNT(DISTINCT S.SOID) AS TotalOrders, SUM(S.Total) AS TotalSpent FROM TVendor V JOIN TPurchaseOrder P ON V.VendID = P.VendID Join TEmployee E ON P.EmpID=E.EmpID JOIN TSalesOrder S ON S.SOID = S.SOID JOIN TDeliveryOut D ON E.EmpID = D.EmpIDShip</pre> % Results Messages <table><thead><tr><th>VendID</th><th>VendorName</th><th>ShippingType</th><th>TotalOrders</th><th>TotalSpent</th></tr></thead><tbody><tr><td>17002</td><td>Vitae Orci PC</td><td>Ground</td><td>300</td><td>157113.00</td></tr><tr><td>17003</td><td>Metus Facilisis Ltd</td><td>Standard</td><td>300</td><td>471339.00</td></tr><tr><td>17008</td><td>Adipiscing Fringilla Porttitor Foundation</td><td>Standard</td><td>300</td><td>785565.00</td></tr><tr><td>17006</td><td>Velit Ltd</td><td>USPS Overnight</td><td>300</td><td>1728243.00</td></tr><tr><td>17012</td><td>Malesuada Vel Foundation</td><td>Ground</td><td>300</td><td>942678.00</td></tr><tr><td>17010</td><td>Nec Orci Industries</td><td>Standard</td><td>300</td><td>471339.00</td></tr><tr><td>17008</td><td>Adipiscing Fringilla Porttitor Foundation</td><td>USPS Overnight</td><td>300</td><td>471339.00</td></tr><tr><td>17010</td><td>Nec Orci Industries</td><td>USPS Overnight</td><td>300</td><td>471339.00</td></tr><tr><td>17011</td><td>Arcu Vestibulum Inc.</td><td>Ground</td><td>300</td><td>628452.00</td></tr><tr><td>17005</td><td>Et Company</td><td>Standard</td><td>300</td><td>628452.00</td></tr><tr><td>17009</td><td>Commodo Tincidunt Nibh Institute</td><td>Ground</td><td>300</td><td>785565.00</td></tr><tr><td>17003</td><td>Metus Facilisis Ltd</td><td>USPS Overnight</td><td>300</td><td>157113.00</td></tr><tr><td>17007</td><td>Nullam Velit Dui Incorporated</td><td>Standard</td><td>300</td><td>314226.00</td></tr><tr><td>17005</td><td>Et Company</td><td>USPS Overnight</td><td>300</td><td>1728243.00</td></tr><tr><td>17004</td><td>Felis Enet Varius LLP</td><td>Ground</td><td>300</td><td>471339.00</td></tr></tbody></table>	VendID	VendorName	ShippingType	TotalOrders	TotalSpent	17002	Vitae Orci PC	Ground	300	157113.00	17003	Metus Facilisis Ltd	Standard	300	471339.00	17008	Adipiscing Fringilla Porttitor Foundation	Standard	300	785565.00	17006	Velit Ltd	USPS Overnight	300	1728243.00	17012	Malesuada Vel Foundation	Ground	300	942678.00	17010	Nec Orci Industries	Standard	300	471339.00	17008	Adipiscing Fringilla Porttitor Foundation	USPS Overnight	300	471339.00	17010	Nec Orci Industries	USPS Overnight	300	471339.00	17011	Arcu Vestibulum Inc.	Ground	300	628452.00	17005	Et Company	Standard	300	628452.00	17009	Commodo Tincidunt Nibh Institute	Ground	300	785565.00	17003	Metus Facilisis Ltd	USPS Overnight	300	157113.00	17007	Nullam Velit Dui Incorporated	Standard	300	314226.00	17005	Et Company	USPS Overnight	300	1728243.00	17004	Felis Enet Varius LLP	Ground	300	471339.00
VendID	VendorName	ShippingType	TotalOrders	TotalSpent																																																																															
17002	Vitae Orci PC	Ground	300	157113.00																																																																															
17003	Metus Facilisis Ltd	Standard	300	471339.00																																																																															
17008	Adipiscing Fringilla Porttitor Foundation	Standard	300	785565.00																																																																															
17006	Velit Ltd	USPS Overnight	300	1728243.00																																																																															
17012	Malesuada Vel Foundation	Ground	300	942678.00																																																																															
17010	Nec Orci Industries	Standard	300	471339.00																																																																															
17008	Adipiscing Fringilla Porttitor Foundation	USPS Overnight	300	471339.00																																																																															
17010	Nec Orci Industries	USPS Overnight	300	471339.00																																																																															
17011	Arcu Vestibulum Inc.	Ground	300	628452.00																																																																															
17005	Et Company	Standard	300	628452.00																																																																															
17009	Commodo Tincidunt Nibh Institute	Ground	300	785565.00																																																																															
17003	Metus Facilisis Ltd	USPS Overnight	300	157113.00																																																																															
17007	Nullam Velit Dui Incorporated	Standard	300	314226.00																																																																															
17005	Et Company	USPS Overnight	300	1728243.00																																																																															
17004	Felis Enet Varius LLP	Ground	300	471339.00																																																																															

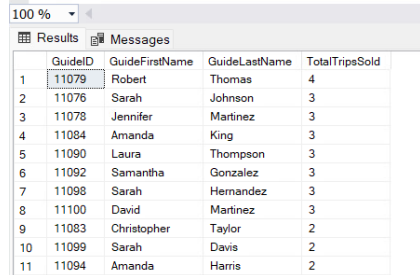
13	Invoice lines for a given sales invoice number and given customer name	<pre>SELECT S.SOID AS InvoiceNumber, SOL.FlyID, F.Name AS ProductName, SOL.Quantity, (F.SalePrice * SOL.Quantity) AS LineTotal, TC.CFirstName AS FirstName, TC.CLastName AS LastName FROM TSalesOrder S JOIN TSalesOrderLine SOL ON S.SOID = SOL.SOID JOIN Tfly F ON SOL.FlyID = F.FlyID JOIN TCustomer TC ON S.CustId = TC.CustID</pre> <i>Saved as a view</i>	<pre>SELECT S.SOID AS InvoiceNumber, SOL.FlyID, F.Name AS ProductName, SOL.Quantity, (F.SalePrice * SOL.Quantity) AS LineTotal, TC.CFirstName AS FirstName, TC.CLastName AS LastName FROM TSalesOrder S JOIN TSalesOrderLine SOL ON S.SOID = SOL.SOID JOIN Tfly F ON SOL.FlyID = F.FlyID JOIN TCustomer TC ON S.CustId = TC.CustID</pre> 100 % Results Messages <table><thead><tr><th></th><th>InvoiceNumber</th><th>FlyID</th><th>ProductName</th><th>Quantity</th><th>LineTotal</th><th>FirstName</th><th>LastName</th></tr></thead><tbody><tr><td>1</td><td>21157</td><td>25016</td><td>Prince Nymph</td><td>99</td><td>594.00</td><td>Cullen</td><td>Arnold</td></tr><tr><td>2</td><td>21195</td><td>25012</td><td>Stimulator</td><td>9</td><td>63.00</td><td>Dane</td><td>Vargas</td></tr><tr><td>3</td><td>21073</td><td>25083</td><td>Humpy</td><td>24</td><td>144.00</td><td>Leah</td><td>Rhodes</td></tr><tr><td>4</td><td>21015</td><td>25026</td><td>Black Gnat</td><td>98</td><td>784.00</td><td>Hyatt</td><td>Sears</td></tr><tr><td>5</td><td>21283</td><td>25063</td><td>CDC Caddis</td><td>9</td><td>81.00</td><td>Sasha</td><td>Pacheco</td></tr><tr><td>6</td><td>21053</td><td>25099</td><td>March Brown</td><td>47</td><td>329.00</td><td>Carter</td><td>Leon</td></tr><tr><td>7</td><td>21053</td><td>25091</td><td>CDC & Elk</td><td>23</td><td>184.00</td><td>Carter</td><td>Leon</td></tr><tr><td>8</td><td>21099</td><td>25023</td><td>Caddis Pupa</td><td>61</td><td>610.00</td><td>Rebecca</td><td>Zamora</td></tr></tbody></table>		InvoiceNumber	FlyID	ProductName	Quantity	LineTotal	FirstName	LastName	1	21157	25016	Prince Nymph	99	594.00	Cullen	Arnold	2	21195	25012	Stimulator	9	63.00	Dane	Vargas	3	21073	25083	Humpy	24	144.00	Leah	Rhodes	4	21015	25026	Black Gnat	98	784.00	Hyatt	Sears	5	21283	25063	CDC Caddis	9	81.00	Sasha	Pacheco	6	21053	25099	March Brown	47	329.00	Carter	Leon	7	21053	25091	CDC & Elk	23	184.00	Carter	Leon	8	21099	25023	Caddis Pupa	61	610.00	Rebecca	Zamora
	InvoiceNumber	FlyID	ProductName	Quantity	LineTotal	FirstName	LastName																																																																				
1	21157	25016	Prince Nymph	99	594.00	Cullen	Arnold																																																																				
2	21195	25012	Stimulator	9	63.00	Dane	Vargas																																																																				
3	21073	25083	Humpy	24	144.00	Leah	Rhodes																																																																				
4	21015	25026	Black Gnat	98	784.00	Hyatt	Sears																																																																				
5	21283	25063	CDC Caddis	9	81.00	Sasha	Pacheco																																																																				
6	21053	25099	March Brown	47	329.00	Carter	Leon																																																																				
7	21053	25091	CDC & Elk	23	184.00	Carter	Leon																																																																				
8	21099	25023	Caddis Pupa	61	610.00	Rebecca	Zamora																																																																				
14	Number of times a discount was applied to a sales order -list all information about the discount, total amount saved by customers that used the discount	<pre>SELECT D.DiscountTypeID, D.DiscountName, COUNT(SOLID) AS TimesApplied, SUM(SOL.Quantity * F.SalePrice) AS TotalAmountSaved FROM TDiscount D JOIN TSalesOrder TSO ON D.DiscountTypeID = TSO.DiscountTypeID JOIN TSalesOrderLine SOL ON TSO.DiscountTypeID = SOL.SOLID JOIN Tfly F ON SOL.FlyID = F.FlyID GROUP BY D.DiscountTypeID, D.DiscountName;</pre> <i>Saved as a view</i>	<pre>SELECT TDiscount.DiscountName, COUNT(*) AS TimesApplied, SUM(TSalesOrder.Total * TDiscount.DiscountPercent / 100) AS TotalAmountSaved FROM TSalesOrder JOIN TDiscount ON TSalesOrder.DiscountTypeID = TDiscount.DiscountTypeID GROUP BY TDiscount.DiscountName;</pre> 100 % Results Messages <table><thead><tr><th></th><th>DiscountName</th><th>TimesApplied</th><th>TotalAmountSaved</th></tr></thead><tbody><tr><td>1</td><td>Christmas Special</td><td>42</td><td>3853.40000000</td></tr><tr><td>2</td><td>Easter Special</td><td>52</td><td>2999.70000000</td></tr><tr><td>3</td><td>Fall Special</td><td>39</td><td>1969.00000000</td></tr><tr><td>4</td><td>Fathers Day Special</td><td>50</td><td>2736.50000000</td></tr><tr><td>5</td><td>Halloween Special</td><td>46</td><td>2082.70000000</td></tr><tr><td>6</td><td>Memorial Day Spe...</td><td>30</td><td>1591.10000000</td></tr><tr><td>7</td><td>New Years Special</td><td>41</td><td>2405.60000000</td></tr></tbody></table>		DiscountName	TimesApplied	TotalAmountSaved	1	Christmas Special	42	3853.40000000	2	Easter Special	52	2999.70000000	3	Fall Special	39	1969.00000000	4	Fathers Day Special	50	2736.50000000	5	Halloween Special	46	2082.70000000	6	Memorial Day Spe...	30	1591.10000000	7	New Years Special	41	2405.60000000																																								
	DiscountName	TimesApplied	TotalAmountSaved																																																																								
1	Christmas Special	42	3853.40000000																																																																								
2	Easter Special	52	2999.70000000																																																																								
3	Fall Special	39	1969.00000000																																																																								
4	Fathers Day Special	50	2736.50000000																																																																								
5	Halloween Special	46	2082.70000000																																																																								
6	Memorial Day Spe...	30	1591.10000000																																																																								
7	New Years Special	41	2405.60000000																																																																								

Three Additional Queries

In addition to the queries that were asked of us by the client, we decided to add a few more that would help in the performance analysis of Elysian. Below are the additional queries, why they may be important and their respective SQL codes and findings.

Query #	Question	Why is this important?	SQL	Partial Output	Recap of Findings
---------	----------	------------------------	-----	----------------	-------------------

1	Average sales in dollars per customer per year	Tracking the average sales per customer per year provides valuable insights into customer behavior, helps in strategic decision-making and enables targeted marketing efforts. It contributes to improving overall business performance and profitability.	<pre>SELECT YEAR(SODate) AS SalesYear, C.CustID, AVG(Total) AS AvgSales FROM TSalesOrder S JOIN TCustomer C ON S.CustID = C.CustID GROUP BY YEAR(SODate), C.CustID;</pre> <i>Saved as a procedure</i>	 Here we can see if customers are buying more or less than they were in the years. For example, Customer 23006 bought the most product/services in 2023 than in 2022 and 2021.
2	Number of customers acquired per year	The number of customers acquired per month is a key performance indicator that directly impacts business growth, profitability, and resource allocation. Monitoring this metric allows businesses to identify trends, optimize marketing	<pre>SELECT YEAR(SODate) AS SalesYear, COUNT(DISTINCT CustID) AS CustomersAcquired FROM TSalesOrder GROUP BY YEAR(SODate);</pre> <i>Saved as a procedure</i>	 The most customers were acquired in 2022, which shows growth of the business and a good customer acquisition strategy.

		and sales efforts, and stay competitive in the market.																																																															
3	Top guides based on the number of trips sold	Identifying top-selling guides based on the number of trips sold is important for evaluating performance, maximizing revenue, enhancing customer satisfaction, optimizing resource allocation, leveraging marketing opportunities, gaining a competitive edge, and driving business growth within the guided trip industry.	SELECT E.EmpID AS GuideID, E.EFirstName AS GuideFirstName, E.ELastName AS GuideLastName, COUNT(GFTID) AS TotalTripsSold FROM TEmployee E JOIN TGuideFishingTrip G ON E.EmpID = G.EmpID GROUP BY E.EmpID, E.EFirstName, E.ELastName ORDER BY TotalTripsSold DESC; <i>Saved as a procedure</i>	<pre>FROM TEmployee E JOIN TGuideFishingTrip G ON E.EmpID = G.EmpID GROUP BY E.EmpID, E.EFirstName, E.ELastName ORDER BY TotalTripsSold DESC;</pre>  <table><thead><tr><th></th><th>GuideID</th><th>GuideFirstName</th><th>GuideLastName</th><th>TotalTripsSold</th></tr></thead><tbody><tr><td>1</td><td>11079</td><td>Robert</td><td>Thomas</td><td>4</td></tr><tr><td>2</td><td>11076</td><td>Sarah</td><td>Johnson</td><td>3</td></tr><tr><td>3</td><td>11078</td><td>Jennifer</td><td>Martinez</td><td>3</td></tr><tr><td>4</td><td>11084</td><td>Amanda</td><td>King</td><td>3</td></tr><tr><td>5</td><td>11090</td><td>Laura</td><td>Thompson</td><td>3</td></tr><tr><td>6</td><td>11092</td><td>Samantha</td><td>Gonzalez</td><td>3</td></tr><tr><td>7</td><td>11098</td><td>Sarah</td><td>Hernandez</td><td>3</td></tr><tr><td>8</td><td>11100</td><td>David</td><td>Martinez</td><td>3</td></tr><tr><td>9</td><td>11083</td><td>Christopher</td><td>Taylor</td><td>2</td></tr><tr><td>10</td><td>11099</td><td>Sarah</td><td>Davis</td><td>2</td></tr><tr><td>11</td><td>11094</td><td>Amanda</td><td>Harris</td><td>2</td></tr></tbody></table>		GuideID	GuideFirstName	GuideLastName	TotalTripsSold	1	11079	Robert	Thomas	4	2	11076	Sarah	Johnson	3	3	11078	Jennifer	Martinez	3	4	11084	Amanda	King	3	5	11090	Laura	Thompson	3	6	11092	Samantha	Gonzalez	3	7	11098	Sarah	Hernandez	3	8	11100	David	Martinez	3	9	11083	Christopher	Taylor	2	10	11099	Sarah	Davis	2	11	11094	Amanda	Harris	2	Guide Robert Thomas has the greatest number of trips sold which places him as the top guide in comparison to other guides
	GuideID	GuideFirstName	GuideLastName	TotalTripsSold																																																													
1	11079	Robert	Thomas	4																																																													
2	11076	Sarah	Johnson	3																																																													
3	11078	Jennifer	Martinez	3																																																													
4	11084	Amanda	King	3																																																													
5	11090	Laura	Thompson	3																																																													
6	11092	Samantha	Gonzalez	3																																																													
7	11098	Sarah	Hernandez	3																																																													
8	11100	David	Martinez	3																																																													
9	11083	Christopher	Taylor	2																																																													
10	11099	Sarah	Davis	2																																																													
11	11094	Amanda	Harris	2																																																													

User Documentation

After the database is finalized, users can access it to run queries that generate insightful reports. Databases offer a structured framework for storing, retrieving, and manipulating data, ultimately enhancing business efficiency and facilitating informed decision-making. The user can access the database through the following steps:

1. Turn on Computer, open browser
2. **Open Walton Virtual Desktop**
 - Go to the Walton College VMWare client website at: <https://waltonlab.uark.edu/>.
 - Select the Access Virtual Desktop with Web Browser



Welcome to the **Walton College of Business** remote computer access portal! You can connect to your virtual desktop by using the VMware Horizon Client or through your web browser.

The VMware Horizon Client offers better performance and features.

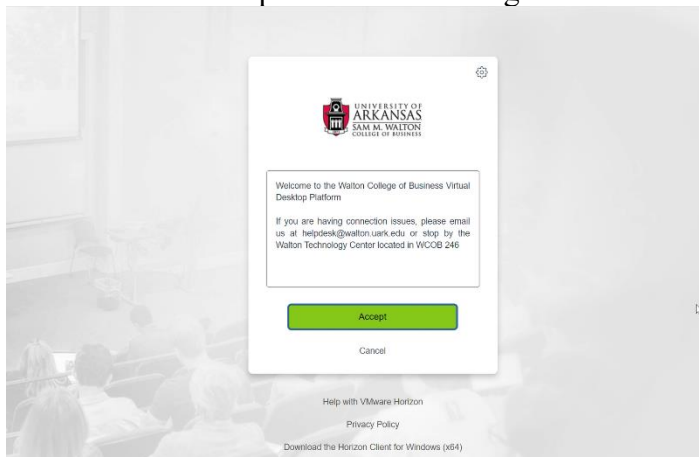
Launch Native Client

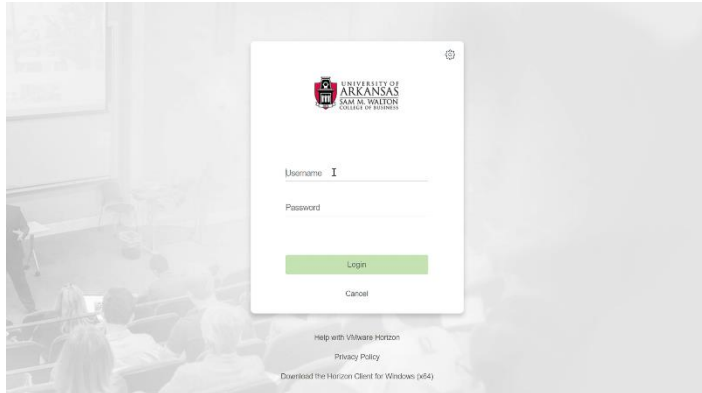
☐ Check here to skip this screen and always use Native Client.

Use your web browser to access your virtual desktop
☐ Check here to skip this screen and always use HTML Access.

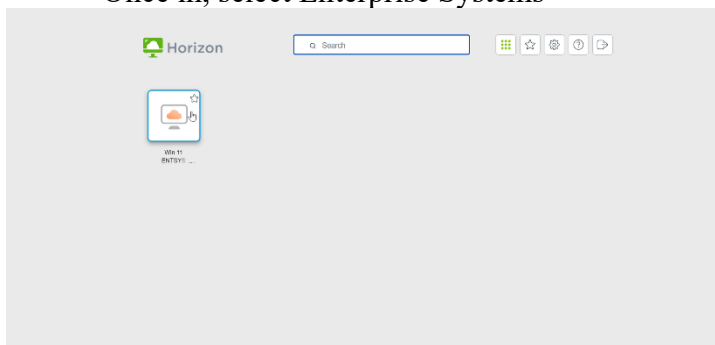
[Click Here to Download VMware Horizon Client](#)

- Click “Accept” and enter the login credentials received





- Complete MFA authentication (through app or phone)
- Once in, select Enterprise Systems

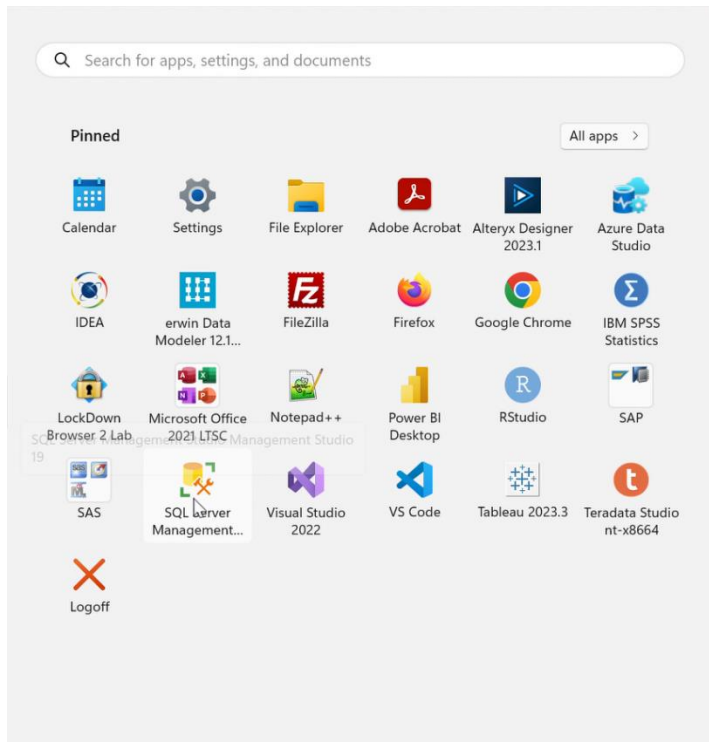


- After a few moments, you will be presented with a windows 10 desktop



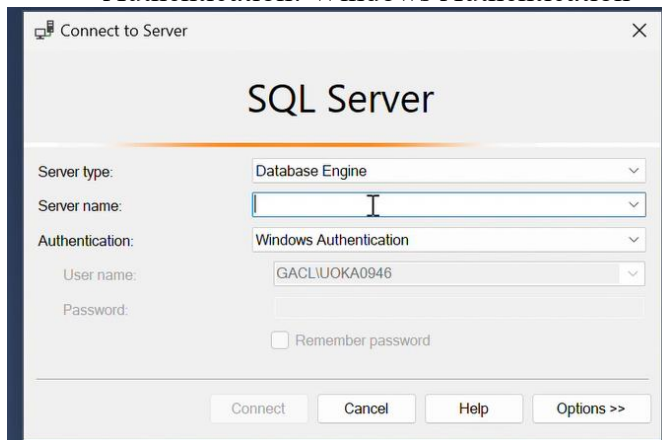
3. Open MSSQL and Sign in

Once In, click on the windows icon and a list of applications should appear, from those select SQL Server Management Studio.



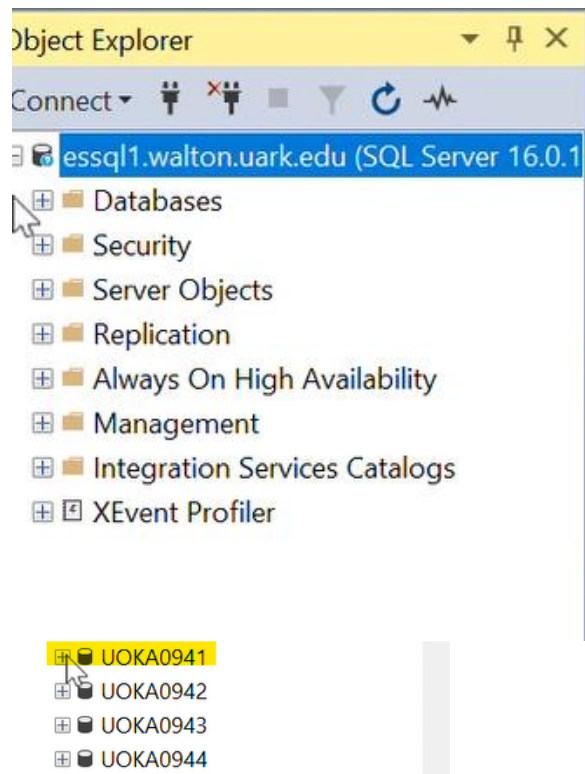
To sign into SQL Server, enter the information below

- Server Type: Database Engine
- Server Name: essql1.walton.uark.edu
- Authentication: Windows Authentication



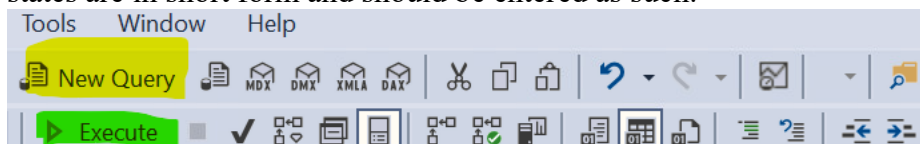
4. Go to your database

Once you have signed into SQL Server, navigate to your database by clicking on Databases. The tab should expand and allow you to scroll through to the database which matches your username.



5. New query

To run queries that will help you analyze your business, click on new query and ‘execute query’.
An example of a query you can run is the number of customers from Oklahoma. Remember that states are in short form and should be entered as such.



6. SQL Code

Below, we will go through the query in the previous step. The query will be entered as:

```
SELECT*
FROM TCUSTOMER
WHERE CCity= 'Rochester';
```

The results from the above query would look something like this:

```

Select*
FROM TCustomer
Where CCity='Rochester';

```

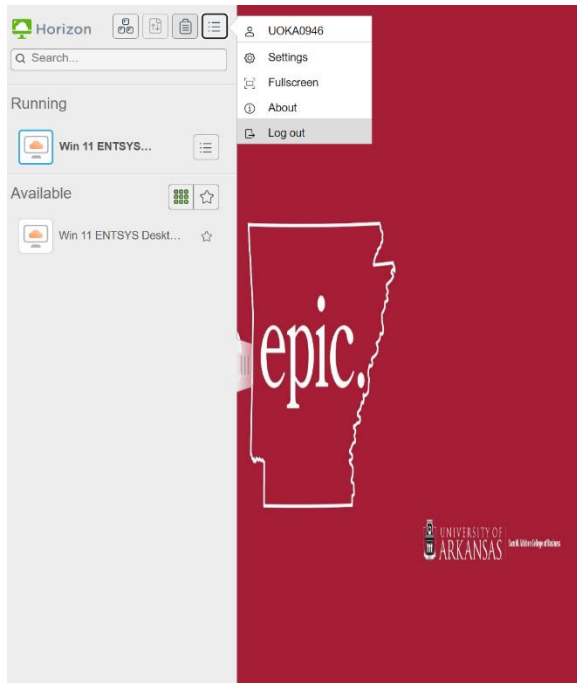
CustID	CustTypeID	CatalogID	StateID	CFirstName	CLastName	CEmail	CAddress	CCity	ZipCode	CPhone
23006	5004	4002	22	Abigail	Pearson	at.pede.cras@google.net	P.O. Box 765, 2853 Dolor Ave	Rochester	83244	(422) 446-4173

Through these findings, we can display all the information about the customer from Rochester.

7. Logging Out

Click on the tab on the right of the screen and log out by clicking the 'log out' option that is displayed when you click on the three lines that indicate a menu.





What We Learned Throughout This Process

Throughout our team project, we have gained vital knowledge that has improved our comprehension of cooperation, problem-solving, and efficient communication. We quickly saw the value of collaboration and that working together with different viewpoints and efforts produces more creative ideas than any one person could on their own. We discovered how to overcome obstacles with resiliency and flexibility, realizing that failures present chances for development and education. Furthermore, our initiative demonstrated the value of open and honest communication, which allowed us to coordinate our efforts, resolve disagreements amicably, and maintain focus on our common objective. Together, we've improved our interpersonal skills, technical expertise, and understanding of the value of cooperation.

Member Name:	What you learned:
Max	Throughout this project I have learned that in the real world not much is going to go your way, and I accepted that. Many times, in life you are going to get knocked down, but pushing through and learning from your mistakes only refines you. During our project there were plenty of times that we thought we had almost everything 100% done and accurate. Well, based on the milestone grades we did not even get half of the stuff correct. With that, we learned to be more creative and start thinking outside of the box. All in all, this project has certainly helped me get a grasp on what the future can hold for me and how to improve myself every day.
Josephine	I gained maximum respect for me and my teammates due to the dedication everyone showed towards ensuring we had our project handed in on time; this showed how much we could do since we worked on everything as a team. At some point during this project, I got really frustrated several times after the milestone 1 and 2 submissions because at first, I didn't really understand what exactly our problem was but after having a second meeting with the client I finally realized our biggest problem stemmed from the ERD.
Haley	Throughout working on this project, I got frustrated several times because after each submission, I was sure that we turned in accurate, quality work. It soon became clear through grading and criticism, that was not the case. I learned that sometimes the outcome you get is not always reflective of the effort you put in and that is okay. Although it was tough receiving a poor grade on some of the submissions after spending countless hours on this project, the criticism that we received was way more valuable to us in the long run than getting a perfect grade on our first try. After each submission, I have become more confident in my database management skills and more satisfied with the work I put in.
Kalen	My participation in this project has been a fantastic learning experience that has really aided in my development both personally and professionally during the semester. Among the most important things I've learnt is how important collaboration and teamwork are. By collaborating closely with my classmates, I was able to understand a variety of viewpoints, discover how to best utilize each person's unique skills, and see the importance of teamwork in accomplishing shared objectives. This project also gave me practical

	experience in project management, from preparation and execution to oversight and modification. I discovered how to overcome obstacles with fortitude, flexibility, and problem-solving abilities, understanding that failures present chances for growth and development.
Chisanshi	I gained an appreciation for teamwork through this project, working in it made me realize how much can be achieved if you work with a team towards a certain goal. I also learned about the value of constructive criticism and how it makes way for improvement if used well. This project also made me realize that perfection is rare and all that matters are consistent efforts that will eventually lead to progress and success.

Team Functions/Insights

Throughout the assignment, our team showed excellent cooperation and communication. Positive interactions among team members created a supportive environment. Regular meetings made it possible to allocate tasks and monitor progress effectively, and accessibility and openness were ensured through a combination of face-to-face meetings and online meetings. Even while there were occasionally difficulties such as different viewpoints on certain tasks, they were quickly resolved by direct communication and compromise. All in all, the team worked well together to accomplish its objectives via clear communication and excellent cooperation.

Over the course of the project, we discovered several helpful insights that greatly advanced our comprehension of the topic. We discovered a variety of viewpoints and ways to problem-solving by closely interacting with team members, which enhanced our analytical and decision-making procedures. In addition, having conversations and meetings with the Professor or TA enabled us to pinpoint important objectives, difficulties, and openings within the project's parameters. We improved our techniques and methods through data-driven assessments and repeated feedback loops, which produced more significant and well-informed results. All things considered, these insights not only improved the results of our project but also offered insightful advice and best practices for similar projects in the future.

Appendix

Team Contract

Welcome to our team contract, a fundamental agreement outlining the values, standards, and pledges directing our group's performance. It is critical that we have a mutual understanding of how we will cooperate, communicate, and accomplish our shared objectives before we start this cooperative project. This agreement acts as a guide for building a climate of mutual respect, responsibility, and cooperation among team members, enabling them to individually bring special skills and viewpoints to bear on achieving our common goals. We pledge to maintain the standards and values stated in this contract to foster a welcoming and inclusive work environment where each team member may reach their full potential. We set out on this adventure together, committed to achieving success for both of us and greatness.

Team Name: Crimson and Cream

Logo:



Team Motto: Achieve as one excel as a team

Team Members

Name	Email	Phone	Strengths	Availability to Meet
Josephine Nyachwo	Josephine.g.nyachwo-1@ou.edu	(405)-318-7729	Flexibility	Friday 12pm-1pm
Chisanshi Chiwele	Chisanshi.chiwele-1@ou.edu	(405)-839-1843	Adaptability	Friday 12pm-1pm

Max Gottwald	maximiliangottwald@ou.edu	(727)-255-9964	Open minded	Friday 12pm-1pm
Haley Raburn	Raburnhaley@ou.edu	(918)-398-3560	Communication	Friday 12pm-1pm
Kalen Lee	Klee@ou.edu	(580)-490-2575	Talkative	Friday 12pm-1pm

Team Expectations for the confidential peer evaluation :

Communication
TeamWork
Attendance
Punchtuality

The behavior for which points will be deducted on the confidential peer evaluation:

Late Submission
No Communication
Poor Effort
Low Participation

Data Dictionary Model

Table	Field Name	Key	Data Type	Size	Null	References	Sample
TState	StateID	PK	int (auto increment)		not null		0-50
	StateName		varchar	2	not null		CA
TColor	ColorID	PK	int (auto increment)		not null		1055-1078
	CName		varchar	25	not null		Pink
TSize	SizeID	PK	int (auto increment)		not null		2001-2018
	SName		varchar	25	not null		16
TPattern	PatternID	PK	int (auto increment)		not null		3222-3402
	PatternName		varchar	25	not null		Adam's Parachute
	Description		varchar	100	not null		
TAnnualCatalog	CatalogID	PK	int (auto increment)		not null		4001-5000
	ACName		varchar	25	not null		
TCustomerType	CustTypeID	PK	int (auto increment)		not null		5001-5004
	CTName		varchar	25	null allowed		The One-Off Tourist
TEmployeeType	EmployeeTypeID	PK	int (auto increment)		not null		6001-6015
	EmployeeTypeName		varchar	25	not null		Accounts Receivable
	IsAccountReceivable(Y/N)		varchar	1	not null		Y
	IsAccountPayable(Y/N)		varchar	1	not null		Y
TPaymentMethod	PaymentMethodTypeID	PK	int (auto increment)		not null		7001-7003
	PaymentMethodName		varchar	25	not null		Card

Table	Field Name	Key	Data Type	Size	Null	References	Sample
TShippingType	STID	PK	int (auto increment)		not null		8001-8004
	STName		varchar	25	not null		Overnight
TGuideSkill	GSID	PK	int (auto increment)		not null		9001-10000
	GSName		varchar	25	not null		Backpacking skills
	KnownRoutes		varchar	100	not null		
TDiscount	DiscountTypeID	PK	int (auto increment)		not null		10001-1012
	DiscountDate		date		not null		9/23/2020
	DiscountName		varchar	25	not null		Black Friday Deal
	DiscountPercent		decimal	4	not null		40%
TSalesOrderChannel	SOCID	PK	int (auto increment)		not null		27000-27002
	Channel Name		varchar	25	not null		In Person
TEmployee	EmpID	PK	int (auto increment)		not null		11001-11102
	EmployeeTypeID	FK	int		not null	TEmployeeType	
	IsGuide(Y/N)		varchar	1	not null		Y
	EFirstName		varchar	50	not null		John
	ELastName		varchar	50	not null		Doe
	EEmail		varchar	50	not null		john.doe@gmail.com
	EAddress		varchar	100	not null		12345 Mockingbird Ln.
	City		varchar	50	not null		San Diego
	ZipCode		varchar	5	not null		73069
	Phone		varchar	14	not null		(918)-576-9362
T<PurchaseOrderLine>	POLID	PK	int (auto increment)		not null		12001-13000
	POID	FK	int		not null	TPurchaseOrder	

Table	Field Name	Key	Data Type	Size	Null	References	Sample
	POLQuantity		int		not null		6
TPaymentIn	PayInID	PK	int (auto increment)		not null		13001-14000
	EmpID	FK	int		not null	TEmployee	
	PIDate		date		not null		9/23/2020
	Amount		money		not null		\$45
	PaymentMethodTypeID	FK	int		not null	TPaymentMethod	
T<SalesOrderLine>	SOLID	PK	int (auto increment)		not null		14001-15000
	FlyID	FK	int		not null	TFly	
	SOID	FK	int		not null	TSalesOrder	
	Quantity		int		not null		6
	SOLStatus		varchar	100	not null		Shipped
T<DIY>	DIYID	PK	int (auto increment)		not null		15001-16000
	FlyID-Kit	FK	int		not null	TFly	
	FlyID-Part	FK	int		not null	TFly	
	Quantity		int		not null		6
TPaymentOut	PayOutID	PK	int (auto increment)		not null		16001-16102
	PaymentMethodTypeID	FK	int		not null	TPaymentMethod	
	EmpID	FK	int		not null	TEmployee	
	Invoice#		int		not null		
	Amount		money		not null		\$45
	PODate		date		not null		9/23/2020
TVendor	VendID	PK	int (auto increment)		not null		17001-17102
	EmpID	FK	int		not null	TEmployee	
	VName		varchar	25	not null		

Table	Field Name	Key	Data Type	Size	Null	References	Sample
	VendorAccount#		int		not null		
	Terms		varchar		not null		
	VDate		date		not null		9/23/2020
	VStreet		varchar	100	not null		12345 Mockingbird Ln.
	VCity		varchar	50	not null		San Diego
	StateID	FK	int	2	not null	TState	CA
	VZipCode		varchar	5	not null		73069
	VEmail		varchar	50	not null		artisans-12@gmail.com
	VPhone		varchar	14	not null		(918)-576-9362
	ContactName		varchar	50	not null		John
TDeliveryIn	DelIID	PK	int (auto increment)		not null		18001-19000
	EmpID-Deliver	FK	int		not null	TEmployee	
	EmpID-Inspect	FK	int		not null	TEmployee	
	EmpID-Receive	FK	int		not null	TEmployee	
	DIDate		date		not null		9/23/2020
TDeliveryOut	DelOID	PK	int (auto increment)		not null		19001-20000
	STID	FK	int		not null	TShippingType	
	EmpID-Pick	FK	int		not null	TEmployee	
	EmpID-Pack	FK	int		not null	TEmployee	
	EmpID-Ship	FK	int		not null	TEmployee	
	TrackingNumber		int		not null		
	ShipDate		date		not null		9/23/2020
	Cost		money		not null		\$10
TGuideFishingTrip	GFTID	PK	int (auto increment)		not null		20001-21000

Table	Field Name	Key	Data Type	Size	Null	References	Sample
	SOID	FK	int		not null	TSalesOrder	
	GSID	FK	int		not null	TGuideSkill	
	EmpID	FK	int		not null	TEmployee	
	MaxPartySize		int		not null		25
	GroupQuantity		int		not null		10
	TripTotal		money		not null		\$200
TSalesOrder	SOID	PK	int (auto increment)		not null		21001-22000
	PayInID	FK	int		not null	TPaymentIn	
	CustID	FK	int		not null	TCustomer	
	EmpID	FK	int		not null	TEmployee	
	DiscountTypeID	FK	int		not null	TDiscout	
	SOCID	FK	int		not null	SalesOrderChannel	
	SODate		date		not null		9/23/2020
	Total		money		not null		\$45
TPurchaseOrder	POID	PK	int (auto increment)		not null		22001-22102
	EmpID	FK	int		not null	TEmployee	
	PayOutID	FK	int		not null	TPaymentOut	
	VendID	FK	int		not null	TVendor	
	Date		date		not null		9/23/2020
TCustomer	CustID	PK	int (auto increment)		not null		23001-24000
	CustTypeID	FK	int		not null	TCustomerType	
	CatalogID	FK	int		not null	TAnnualCatalog	
	CFirstName		varchar	50	not null		John
	CLastName		varchar	50	not null		Doe
	CEmail		varchar	50	not null		john.doe@gmail.com
	CPhone		varchar	14	not null		

Elysian Fly Case

Crimson and Cream

Table	Field Name	Key	Data Type	Size	Null	References	Sample
	CAddress		varchar	100	not null		12345 Mockingbird Ln.
	CCity		varchar	50	not null		San Diego
	StateID	FK	int	2	not null	TState	CA
	ZipCode		varchar	5	not null		73069
TFly	FlyID	PK	int (auto increment)		not null		25001-26000
	ColorID	FK	int		not null	TColor	
	VendID	FK	int		not null	TVendor	
	Name		varchar	25	not null		
	SalePrice		money		not null		\$10
	Qty_OH		int		not null		100
	SizeID	FK	int		not null	TSize	
	ReorderPoint		varchar		not null		50
	PatternID	FK	int		not null	TPattern	
	PurchaseCost		money		not null		\$10

Project Management

Project Start Date	3/8/2024	Project End Date	4/28/2024	Cost (per 60 min)	
	Student Name	Duration (Min)	% Complete	Subtotal Minutes	Subtotal Cost
Milestone 1					
Read Case + Prepare Questions for client	Haley	10	100%	10	\$4
Read Case + Prepare Questions for client	Max	10	100%	10	\$4
Read Case + Prepare Questions for client	Chisanshi	10	100%	10	\$4
Read Case + Prepare Questions for client	Josephine	10	100%	10	\$4
Read Case + Prepare Questions for client	Kalen	10	100%	10	\$4
Client Meeting	Entire Team	15	100%	15	\$6
ERD Design	Haley	180	100%	180	\$75
ERD Design	Max	180	100%	180	\$75
ERD Design	Chisanshi	180	100%	180	\$75
ERD Design	Josephine	180	100%	180	\$75
ERD Design	Kalen	300	100%	300	\$125
Assumptions	Entire Team	20	100%	20	\$8
Write-up preparation	Haley	90	100%	90	\$38
Write-up preparation	Max	90	100%	90	\$38
Write-up preparation	Chisanshi	90	100%	90	\$38
Write-up preparation	Josephine	90	100%	90	\$38
Write-up preparation	Kalen	90	100%	90	\$38
Sub Total				1555	\$648
Milestone 2					
Update ERD Design	Haley	60	100%	60	\$25
Update ERD Design	Max	60	100%	60	\$25

*Elysian Fly Case**Crimson and Cream*

Update ERD Design	Chisanshi	60	100%	60	\$25
Update ERD Design	Josephine	60	100%	60	\$25
Update ERD Design	Kalen	60	100%	60	\$25
Update Template	Haley	90	100%	90	\$38
Update Template	Max	90	100%	90	\$38
Update Template	Chisanshi	90	100%	90	\$38
Update Template	Josephine	90	100%	90	\$38
Update Template	Kalen	90	100%	90	\$38
Sub Total				750	\$313
Milestone 3					
Update ERD Design	Haley	120	100%	120	\$50
Update ERD Design	Max	120	100%	120	\$50
Update ERD Design	Chisanshi	120	100%	120	\$50
Update ERD Design	Josephine	120	100%	120	\$50
Update ERD Design	Kalen	120	100%	120	\$50
Update Template	Haley	180	100%	180	\$75
Update Template	Max	180	100%	180	\$75
Update Template	Chisanshi	180	100%	180	\$0
Update Template	Josephine	180	100%	180	\$75
Update Template	Kalen	180	100%	180	\$75
Data Dictionary	Haley	60	100%	60	\$25
Data Dictionary	Max	60	100%	60	\$25
Data Dictionary	Chisanshi	60	100%	60	\$25
Data Dictionary	Josephine	60	100%	60	\$25
Data Dictionary	Kalen	60	100%	60	\$25
Physical Design & Implementation	Haley	180	100%	180	\$75
Physical Design & Implementation	Max	180	100%	180	\$75
Physical Design & Implementation	Chisanshi	180	100%	180	\$75

*Elysian Fly Case**Crimson and Cream*

Physical Design & Implementation	Josephine	180	100%	180	\$75
Physical Design & Implementation	Kalen	180	100%	180	\$75
Sub Total				2700	\$1,125
Final Submission					
Update ERD Design	Haley	15	100%	15	\$6
Update ERD Design	Max	15	100%	15	\$6
Update ERD Design	Chisanshi	15	100%	15	\$6
Update ERD Design	Josephine	15	100%	15	\$6
Update ERD Design	Kalen	15	100%	15	\$6
Update Template	Haley	60	100%	60	\$25
Update Template	Max	60	100%	60	\$25
Update Template	Chisanshi	60	100%	60	\$25
Update Template	Josephine	60	100%	60	\$25
Update Template	Kalen	60	100%	60	\$25
Data Dictionary	Haley	15	100%	15	\$6
Data Dictionary	Max	15	100%	15	\$6
Data Dictionary	Chisanshi	15	100%	15	\$6
Data Dictionary	Josephine	15	100%	15	\$6
Data Dictionary	Kalen	15	100%	15	\$6
Physical Design & Implementation	Haley	180	100%	180	\$75
Physical Design & Implementation	Max	180	100%	180	\$75
Physical Design & Implementation	Chisanshi	180	100%	180	\$75
Physical Design & Implementation	Josephine	180	100%	180	\$75
Physical Design & Implementation	Kalen	180	100%	180	\$75
User Documentation	Haley	30	100%	30	\$13
User Documentation	Max	30	100%	30	\$13
User Documentation	Chisanshi	30	100%	30	\$13
User Documentation	Josephine	30	100%	30	\$13
User Documentation	Kalen	30	100%	30	\$13

Elysian Fly Case

Crimson and Cream

Sub Total		1500	\$625
Total		6505	\$2,710